

DISTRIBUTED SYSTEMS

Lab 11

Nuno Preguiça, Sérgio Duarte, Kevin Gallagher,
Salvatore Signorello, João Bordalo, Dina Borrego

GOALS

By the end of this lab, you should be able to:

- In relation to Kafka
 - Understand the abstractions provided by Kafka
 - Understand how to publish events in Java
 - Understand how to subscribe event in Java
 - Understand the asynchronous processing of Kafka received operations in the context of replication in the project.
- Zookeeper
 - Understand the abstractions provided by Zookeeper
 - Understand how to create a tuple (znode) in Java
 - Understand how to monitor a tuple (znode) in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- Overall Revision of the Project Goals

GOALS

By the end of this lab, you should be able to:

- **In relation to Kafka**
 - **Understand the abstractions provided by Kafka**
 - Understand how to publish events in Java
 - Understand how to subscribe event in Java
 - Understand the asynchronous processing of Kafka received operations in the context of replication in the project.
- Zookeeper
 - Understand the abstractions provided by Zookeeper
 - Understand how to create a tuple (znode) in Java
 - Understand how to monitor a tuple (znode) in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- Understand the implementation path for the different implementation alternatives for Replication in the second project.

KAFKA: PUBLISH/SUBSCRIBE

Apache Kafka is a popular open-source distributed event streaming platform.

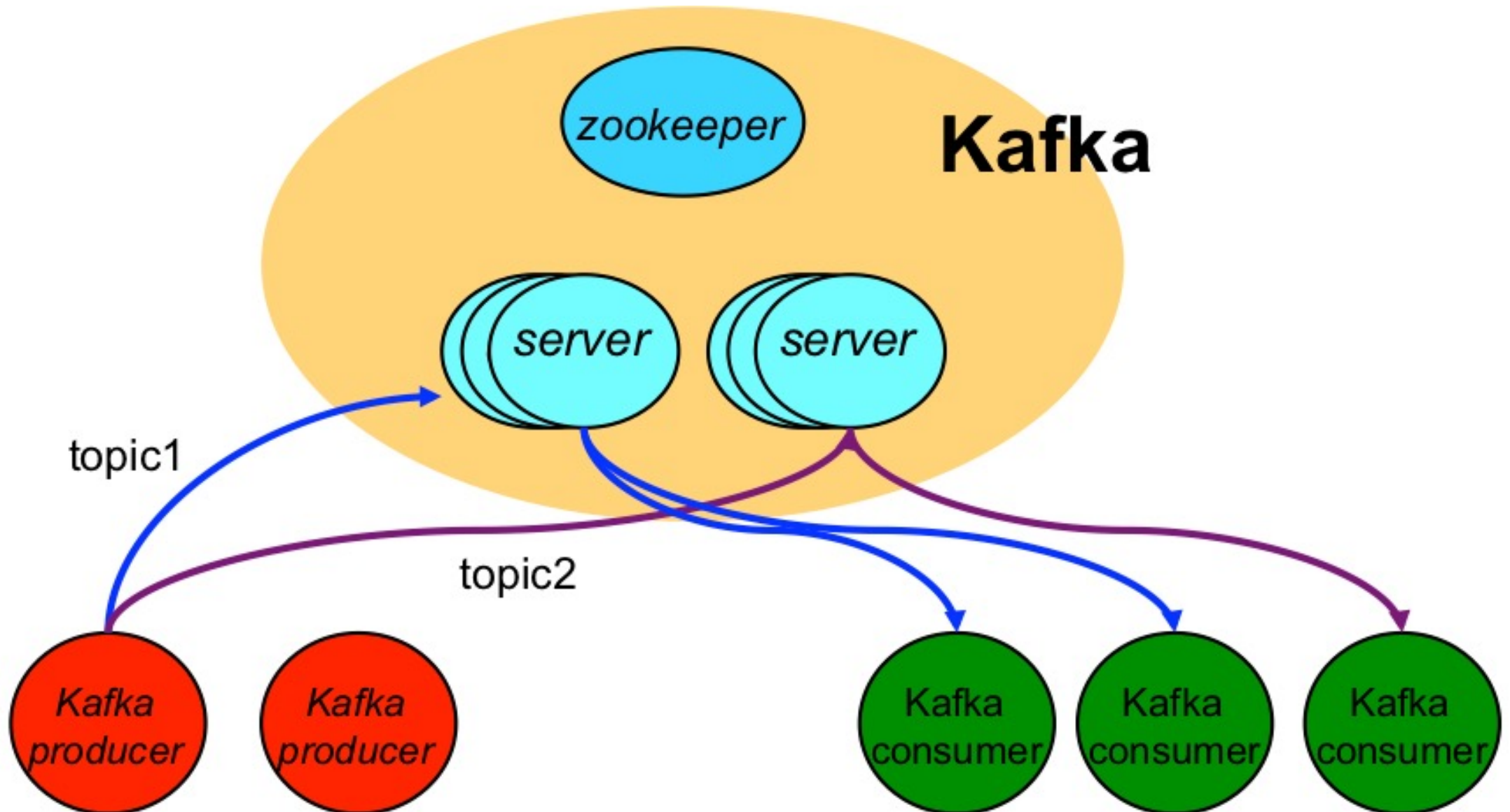
It provides indirect and reliable communication through a publish/subscribe API

There are two main entities in this context:

- Publisher/Producer: Emits events/messages on a given topic
- Subscriber/Consumer: Registers interest in receiving events/messages that are published on a given topic (or set of topics)

Interactions in this model are independent of space and time, meaning that two entities might exchange information without knowing each other or without being online simultaneously.

KAFKA: PUBLISH/SUBSCRIBE



KAFKA: RELIABILITY

In Apache Kafka events that are propagated through it (i.e., published) are stored in a persistent log (i.e., on disk).

- The log survives to crashes of servers/brokers
- The log is pruned in a lazy fashion (by configuration)
- The log can be replayed from the beginning to receive events published while the subscriber (i.e., receiver) was offline.
- Each topic has its own, fully independent, log of events.

KAFKA: ORDER OF EVENTS

For performance reasons, for each topic, the event log can be partitioned across distinct groups of server.

For each topic, events are totally ordered within each partition.

The event record offset defines the order of the events within the partition of the topic where a given event is located.

Topics that rely on a single partition have a total order defined for all receivers.

For topics that use more than one partition, events are partially ordered.

KAFKA: PUBLISH/SUBSCRIBE

In the context of the project, one of the variants of replication that you can implement will use Kafka to propagate operations received from a (end-user) client or from another replicated Messages service to all replicas in the same domain.

This will be achieved by publishing the received operations as events through Kafka in a topic (that can be selected by you, but you need to guarantee that Messages servers in different domains will use different topics).

Note that interactions between your servers and Kafka are asynchronous.

KAFKA: PROGRAMMING (PRELIMINARIES)

There is a Maven Java Project in the course web page that has the examples presented in this lab (both kafka and zookeeper)

As usual the zip and project name is: **lab11**

KAFKA: PROGRAMMING (PRELIMINARIES)

You need to add maven dependencies to use Kafka in your project (file pom.xml):

In the <dependencies> section:

```
<dependency>  
    <groupId>org.apache.kafka</groupId>  
    <artifactId>kafka-clients</artifactId>  
    <version>4.2.0</version>  
</dependency>
```

KAFKA: PROGRAMMING (PRELIMINARIES)

To use and interact with Kafka you will need the Kafka server (that internally also uses Zookeeper) to be running.

In the project you have a script file that runs a docker container with Kafka (and Zookeeper) ready for use in either your local machine (address localhost) or accessible within a docker file in the sd-net (under the name kafka):

start-kafka.sh [localhost | kafka]

KAFKA: PROGRAMMING (PRELIMINARIES)

To use it on your local machine (for instance to test your code locally):

```
sh start-kafka.sh localhost
```

```
nmp@Mac lab11 % sh start-kafka.sh localhost
Error response from daemon: network with name sdnet already exists
Using default tag: latest
latest: Pulling from smduarte/sd2324-kafka
Digest: sha256:fcf298569dac8c405ecc4a17895463a28f69b20de5575287b925f85b8297e861
Status: Image is up to date for smduarte/sd2324-kafka:latest
docker.io/smduarte/sd2324-kafka:latest
Launching Kafka Server: localhost
Error response from daemon: No such container: kafka
2026-04-29 22:15:05,342 CRIT Supervisor is running as root. Privileges were not dropped because no user is specified in the config file. If you intend to run as root, you can set user=root in the config file to avoid this message.
2026-04-29 22:15:05,342 INFO Included extra file "/etc/supervisor/conf.d/kafka.conf" during parsing
2026-04-29 22:15:05,342 INFO Included extra file "/etc/supervisor/conf.d/zookeeper.conf" during parsing
2026-04-29 22:15:05,344 INFO RPC interface 'supervisor' initialized
2026-04-29 22:15:05,344 CRIT Server 'unix_http_server' running without any HTTP authentication checking
2026-04-29 22:15:05,344 INFO supervisord started with pid 8
2026-04-29 22:15:06,351 INFO spawned: 'kafka' with pid 9
2026-04-29 22:15:06,353 INFO spawned: 'zookeeper' with pid 10
2026-04-29 22:15:07,390 INFO success: kafka entered RUNNING state, process has stayed up for > than 1 seconds (startsecs)
2026-04-29 22:15:07,390 INFO success: zookeeper entered RUNNING state, process has stayed up for > than 1 seconds (startsecs)
```

To use it with your containers in docker:

```
sh start-kafka.sh kafka
```

GOALS

By the end of this lab, you should be able to:

- **In relation to Kafka**
 - Understand the abstractions provided by Kafka
 - **Understand how to publish events in Java**
 - Understand how to subscribe event in Java
 - Understand the asynchronous processing of Kafka received operations in the context of replication in the project.
- Zookeeper
 - Understand the abstractions provided by Zookeeper
 - Understand how to create a tuple (znode) in Java
 - Understand how to monitor a tuple (znode) in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- Overall Revision of the Project Goals

KAFKA: PROGRAMMING (PUBLISH EVENTS)

To show how you can publish an event on a given topic, we have provided a simple application (main) that serves only as an example of how you can publish events.

In your project, your Messages servers are going to be responsible for publishing events (operations) and there is no need for new classes with a main for this purpose.

The Javadoc of Kafka (for version 4.2.0) can be found at:

<https://kafka.apache.org/42/javadoc/index.html>

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaSendMsg

```
public class KafkaSendMsg
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide the topic name:");
        String topic = sc.nextLine().trim();

        System.out.println("Provide the message to send:");
        String value = sc.nextLine().trim();

        sc.close();

        KafkaUtils.createTopic(topic); //Attempts to create a topic (might already exist)
        KafkaPublisher publisher = KafkaPublisher.createPublisher("localhost:9092, kafka:9092");
        long offset = publisher.publish(topic, value);
        if(offset >= 0) {
            System.out.println("Message published with offset (sequence number): " + offset);
        } else {
            System.out.println("Failed to publish message");
        }
    }
}
```

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaSendMsg

```
public class KafkaSendMsg
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide the topic name:");
        String topic = sc.nextLine().trim();

        System.out.println("Provide the message to send:");
        String value = sc.nextLine().trim();

        sc.close();

        KafkaUtils.createTopic(topic); //Attempts to create a topic (might already exist)
        KafkaPublisher publisher = KafkaPublisher.createPublisher("localhost:9092, kafka:9092");
        long offset = publisher.publish(topic, value);
        if(offset >= 0) {
            System.out.println("Message published with offset (sequence number): " + offset);
        } else {
            System.out.println("Failed to publish message");
        }
    }
}
```

This is simple code for asking the user to provide the name of the topic where we are going to publish the message as well as the message itself

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaSendMsg

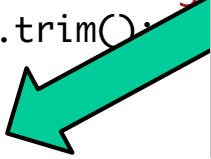
```
public class KafkaSendMsg
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide the topic name");
        String topic = sc.nextLine().trim();

        System.out.println("Provide the message");
        String value = sc.nextLine().trim();

        sc.close();

        KafkaUtils.createTopic(topic); //Attempts to create a topic (might already exist)
        KafkaPublisher publisher = KafkaPublisher.createPublisher("localhost:9092, kafka:9092");
        long offset = publisher.publish(topic, value);
        if(offset >= 0) {
            System.out.println("Message published with offset (sequence number): " + offset);
        } else {
            System.out.println("Failed to publish message");
        }
    }
}
```



This is using our own auxiliary class to create a topic with the requested name (the operation might fail if the topic already exists, but independently of that we will proceed).

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaSendMsg

```
public class KafkaSendMsg
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide topic name:");
        String topic = sc.nextLine();

        System.out.println("Provide message value:");
        String value = sc.nextLine();

        sc.close();

        KafkaUtils.createTopic(topic); //Attempts to create a topic (might already exist)
        KafkaPublisher publisher = KafkaPublisher.createPublisher("localhost:9092, kafka:9092");
        long offset = publisher.publish(topic, value);
        if(offset >= 0) {
            System.out.println("Message published with offset (sequence number): " + offset);
        } else {
            System.out.println("Failed to publish message");
        }
    }
}
```

We create a KafkaPublisher (we will see the code in the following slides) using the static method createPublisher. Its argument is a string with possible locations for the kafka servers separated by commas (hostname:port)



KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaSendMsg

```
public class KafkaSendMsg
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide Topic Name:");
        String topic = sc.nextLine();

        System.out.println("Provide Message Value:");
        String value = sc.nextLine();

        sc.close();

        KafkaUtils.createTopic(topic); //Attempts to create topic (might already exist)
        KafkaPublisher publisher = KafkaPublisher.createPublisher("localhost:9092, kafka:9092");
        long offset = publisher.publish(topic, value);
        if(offset >= 0) {
            System.out.println("Message published with offset (sequence number): " + offset);
        } else {
            System.out.println("Failed to publish message");
        }
    }
}
```

We can now publish any number of messages we want. Each message has a topic and a value (the message itself).

This method returns the sequence number of this message in Kafka (notice that sequence numbers are per Topic).



KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaSendMsg

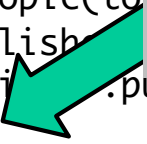
```
public class KafkaSendMsg
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide topic name:");
        String topic = sc.nextLine();

        System.out.println("Provide message value:");
        String value = sc.nextLine();

        sc.close();

        KafkaUtils.createTopic(topic, 1, 1, 1);
        KafkaPublisher publisher = new KafkaPublisher(topic);
        long offset = publisher.publish(topic, value);
        if(offset >= 0) {
            System.out.println("Message published with offset (sequence number): " + offset);
        } else {
            System.out.println("Failed to publish message");
        }
    }
}
```



If the offset (i.e., sequence number) is positive it indicates success; otherwise -1 denotes an error (potentially because the Kafka server could not be contacted). The first event published to a topic has the offset zero.

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (preamble)

```
public class KafkaPublisher {
    static public KafkaPublisher createPublisher(String addr) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Class that can serialize the key format (string)
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        // Class that can serialize the value of events (string)
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        return new KafkaPublisher(new KafkaProducer<String, String>(props));
    }
    private final KafkaProducer<String, String> producer;

    private KafkaPublisher( KafkaProducer<String, String> producer) {
        this.producer = producer;
    }

    public void close() {
        this.producer.close();
    }
}
```

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (preamble)

```
public class KafkaPublisher {  
    static public KafkaPublisher createPublisher(String addr) {  
        Properties props = new Properties();  
  
        // List of pairs hostname:port that allows to contact kafka servers  
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);  
  
        // Class that can serialize the key format (string)  
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());  
  
        // Class that can serialize the value of events (string)  
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());  
  
        return new KafkaPublisher(new KafkaProducer<String, String>(props));  
    }  
  
    private final KafkaProducer<String, String> producer;  
  
    private KafkaPublisher( KafkaProducer<String, String> producer) {  
        this.producer = producer;  
    }  
  
    public void close() {  
        this.producer.close();  
    }  
}
```

The static method used to instantiate this class sets a set of important properties

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (preamble)

```
public class KafkaPublisher {
    static public KafkaPublisher createPublisher(String addr) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Class that can serialize the key format (string)
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        // Class that can serialize the value of events (string)
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        return new KafkaPublisher(new KafkaProducer<String, String>(props));
    }
    private final KafkaProducer<String, String> producer;

    private KafkaPublisher( KafkaProducer<String, String> producer) {
        this.producer = producer;
    }

    public void close() {
        this.producer.close();
    }
}
```

This property indicates the location of Kafka servers, it's a String with pairs hostname:port separated by commas (as seen previously).

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (preamble)

```
public class KafkaPublisher {
    static public KafkaPublisher createPublisher(String addr) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Class that can serialize the key format (string)
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        // Class that can serialize the value of events (string)
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        return new KafkaPublisher(new KafkaProducer<String, String>(props));
    }
    private final KafkaProducer<String, String> producer;

    private KafkaPublisher() {
        this.producer = new KafkaProducer<String, String>(props);
    }

    public void publish(String key, String value) {
        this.producer.send(new ProducerRecord<String, String>(topic, key, value));
    }
}
```

This property indicates the format (particularly, how to serialize) of the Keys of records. Keys are optional when you publish a record in Kafka (only used for partitioning).

Here we assume our keys are Strings.

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (preamble)

```
public class KafkaPublisher {
    static public KafkaPublisher createPublisher(String addr) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Class that can serialize the key format (string)
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        // Class that can serialize the value of events (string)
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        return new KafkaPublisher(new KafkaProducer<String, String>(props));
    }
    private final KafkaProducer<String, String> producer;

    private KafkaPublisher(KafkaProducer<String, String> producer) {
        this.producer = producer;
    }

    public void close() {
        this.producer.close();
    }
}
```

This property indicates the format (particularly, how to serialize) of the values of records (i.e., the content of messages).

Here we assume these are also Strings.

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (preamble)

```
public class KafkaPublisher {  
    static public KafkaPublisher createPublisher(String addr) {  
        Properties props = new Properties();
```

```
        // List of pairs hostname:port t  
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);
```

```
        // Class that can serialize the  
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, "org.apache.kafka.common.serialization.StringSerializer");
```

```
        // Class that can serialize the  
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, "org.apache.kafka.common.serialization.StringSerializer");
```

```
        return new KafkaPublisher(new KafkaProducer<String, String>(props));  
    }  
}
```

```
private final KafkaProducer<String, String> producer;
```

```
private KafkaPublisher(KafkaProducer<String, String> producer) {  
    this.producer = producer;  
}
```

```
public void close() {  
    this.producer.close();  
}
```

This is the KafkaProducer class, which is the class in the Apache Kafka library responsible for managing interactions of a Publisher with the Kafka server(s).

It receives the properties that we prepared before in its constructor.

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (preamble)

```
public class KafkaPublisher {
    static public KafkaPublisher createPublisher(String addr) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Class that can serialize the key format (string)
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, "org.apache.kafka.common.serialization.StringSerializer");

        // Class that can serialize the value
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, "org.apache.kafka.common.serialization.StringSerializer");

        return new KafkaPublisher(new KafkaProducer(props));
    }

    private final KafkaProducer<String, String> producer;

    private KafkaPublisher(KafkaProducer<String, String> producer) {
        this.producer = producer;
    }
}
```

You can terminate a KafkaProducer (if you are not going to publish more messages) by using the close() method.

```
public void close() {
    this.producer.close();
}
```

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (publish methods)

```
// This method is to publish to a topic that relies on partitioning.
public long publish(String topic, String key, String value) {
    try {
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, key, value));
        RecordMetadata rec = promise.get();
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());
        return rec.offset();
    } catch (ExecutionException | InterruptedException x) {
        x.printStackTrace();
        return -1;
    }
}

// This method is to publish to a topic that has no partitioning.
public long publish(String topic, String value) {
    try {
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, value));
        RecordMetadata rec = promise.get();
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());
        return rec.offset();
    } catch (ExecutionException | InterruptedException x) {
        x.printStackTrace();
        return -1;
    }
}
```

Here we have two alternative methods to publish a record to Kafka. They only differ in using or not using a Key associated with that record. The key is used for assigning an event to a partition assuming the topic is partitioned.

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (publish methods)

```
// This method is to publish to a topic that relies on partitioning.
```

```
public long publish(String topic, String key, String value) {
```

```
    try {
```

```
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, key, value));
```

```
        RecordMetadata rec = promise.get();
```

```
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());
```

```
        return rec.offset();
```

```
    } catch (ExecutionException | InterruptedException x) {
```

```
        x.printStackTrace();
```

```
        return -1;
```

```
    }
```

```
}
```

```
// This method is to publish to a topic that has no partitioning.
```

```
public long publish(String topic, String value) {
```

```
    try {
```

```
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, value));
```

```
        RecordMetadata rec = promise.get();
```

```
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());
```

```
        return rec.offset();
```

```
    } catch (Exception x) {
```

```
        x.printStackTrace();
```

```
        return -1;
```

```
    }
```

```
}
```

To publish a Record to Kafka we use the send method of the KafkaProducer. It expects a ProducerRecord argument that is typified with the types of the Key and Value (both String in this case).

The arguments can either be the topic, key, value or only the topic and value.

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (publish methods)

// This method is to publish to a topic that relies on partitioning.

```
public long publish(String topic, String key, String value) {
```

```
    try {
```

```
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, key, value));
```

```
        RecordMetadata rec = promise.get();
```

```
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());
```

```
        return rec.offset();
```

```
    } catch (ExecutionException | InterruptedException x) {
```

```
        x.printStackTrace();
```

```
        return -1;
```

```
    }
```

```
}
```

// This method is to publish to a topic that has no partitioning.

```
public long publish(String topic, String value) {
```

```
    try {
```

```
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, value));
```

```
        RecordMetadata rec = promise.get();
```

```
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());
```

```
        return rec.offset();
```

```
    } catch (ExecutionException | InterruptedException x) {
```

```
        x.printStackTrace();
```

```
        return -1;
```

```
    }
```

```
}
```

The send method returns what is called a Future (i.e., it returns before effectively communicating with the Kafka server).

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (publish methods)

```
// This method is to publish to a topic that relies on partitioning.
public long publish(String topic, String key, String value) {
    try {
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, key, value));
        RecordMetadata rec = promise.get();
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());
        return rec.offset();
    } catch (ExecutionException | InterruptedException x) {
        x.printStackTrace();
        return -1;
    }
}

// This method is to publish to a topic that has no partitioning.
public long publish(String topic, String value) {
    try {
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, value));
        RecordMetadata rec = promise.get();
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());
        return rec.offset();
    } catch (ExecutionException | InterruptedException x) {
        x.printStackTrace();
        return -1;
    }
}
```

The .get() invocation over this Future blocks until we get a reply from the Kafka server (making the method synchronous).

It returns an object with Metadata for the record just created.

KAFKA: PROGRAMMING (PUBLISH EVENTS)

Class KafkaPublisher (publish methods)

// This method is to publish to a topic that relies on partitioning.

```
public long publish(String topic, String key, String value) {  
    try {  
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, key, value));  
        RecordMetadata rec = promise.get();  
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());  
        return rec.offset();  
    } catch (InterruptedException | InterruptedException x) {  
        x.printStackTrace();  
        return -1;  
    }  
}
```

// This method is to publish to a topic that has no partitioning.

```
public long publish(String topic, String value) {  
    try {  
        Future<RecordMetadata> promise = producer.send(new ProducerRecord<String, String>(topic, value));  
        RecordMetadata rec = promise.get();  
        System.out.println("Published to topic " + topic + " with offset " + rec.offset());  
        return rec.offset();  
    } catch (Exception x) {  
        x.printStackTrace();  
        return -1;  
    }  
}
```

The offset method of the RecordMetadata object returned by the get operation provides the offset (i.e., sequence number) of this record in Kafka for this topic. Notice that sequence numbers are per Topic. Consumers consume records in an order that respects this offset (if the topic has a single partition, you can check more about this in the documentation).

GOALS

By the end of this lab, you should be able to:

- **In relation to Kafka**
 - Understand the abstractions provided by Kafka
 - Understand how to publish events in Java
 - **Understand how to subscribe event in Java**
 - Understand the asynchronous processing of Kafka received operations in the context of replication in the project.
- Zookeeper
 - Understand the abstractions provided by Zookeeper
 - Understand how to create a tuple (znode) in Java
 - Understand how to monitor a tuple (znode) in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- Understand the implementation path for the different implementation alternatives for Replication in the second project.

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaReceiver

```
public class KafkaReceiver
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide the topic name:");
        String topic = sc.nextLine().trim();

        sc.close();
        KafkaUtils.createTopic(topic); //Attempts to create a topic (might already exist)
        KafkaSubscriber subscriber = KafkaSubscriber.createSubscriber("localhost:9092, kafka:9092",
                                                                       List.of(topic));

        subscriber.start( new RecordProcessor() {
            @Override
            public void onReceive(ConsumerRecord<String, String> r) {
                System.out.println( r.topic() + " , " + r.offset() + " -> " + r.value());
            }
        });
    }
}
```

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaReceiver

```
public class KafkaReceiver
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide the topic name:");
        String topic = sc.nextLine().trim();

        sc.close();
        KafkaUtils.createTopic(topic); //Attempts to create a topic (might already exist)
        KafkaSubscriber subscriber = KafkaSubscriber.createSubscriber("localhost:9092, kafka:9092",
                                                                       List.of(topic));

        subscriber.start( new RecordProcessor() {
            @Override
            public void onReceive(ConsumerRecord<String, String> r) {
                System.out.println( r.topic() + " , " + r.offset() + " -> " + r.value());
            }
        });
    }
}
```



This is just asking the user for the topic in which this client will be interested.

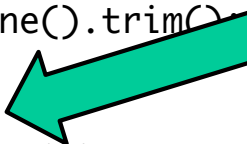
KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

```
public class KafkaReceiver
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide the topic");
        String topic = sc.nextLine().trim();

        sc.close();
        KafkaUtils.createTopic(topic); //Attempts to create a topic (might already exist)
        KafkaSubscriber subscriber = KafkaSubscriber.createSubscriber("localhost:9092, kafka:9092",
                                                                       List.of(topic));

        subscriber.start( new RecordProcessor() {
            @Override
            public void onReceive(ConsumerRecord<String, String> r) {
                System.out.println( r.topic() + " , " + r.offset() + " -> " + r.value());
            }
        });
    }
}
```



Class KafkaReceiver

Like the Producer, we try to create the topic in which we are interested. This operation might fail if the topic was already created before (e.g., by the producer or another subscriber).

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Here we create the subscriber (we will see the code next), as the producer we provide a list of hostname:port pairs that can be used to contact kafka servers. We have created an API where you can also subscribe to topics when creating the KafkaSubscriber.

```
public class  
{  
    public  
    Sca  
  
    Sys  
    Str
```

```
sc.close();  
KafkaUtils.createTopic(topic); //Attempts to create topic (might already exist)  
KafkaSubscriber subscriber = KafkaSubscriber.createSubscriber("localhost:9092, kafka:9092",  
    List.of(topic));  
  
subscriber.start( new RecordProcessor() {  
    @Override  
    public void onReceive(ConsumerRecord<String, String> r) {  
        System.out.println( r.topic() + " , " + r.offset() + " -> " + r.value());  
    }  
});  
}  
}
```



KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaReceiver

```
public class KafkaReceiver
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide the topic name:");
        String topic = sc.nextLine().trim();

        sc.close();
        KafkaUtils.createTopic(topic); //Attempts to create a topic (if it already exist)
        KafkaSubscriber subscriber = KafkaSubscriber.createSubscriber("localhost:9092, kafka:9092",
                                                                       List.of(topic));

        subscriber.start( new RecordProcessor() {
            @Override
            public void onReceive(ConsumerRecord<String, String> r) {
                System.out.println( r.topic() + " , " + r.offset() + " -> " + r.value());
            }
        });
    }
}
```

Since Consumers can be interested in multiple topics, we encapsulate those topics (in this case only one) within a List.



KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaReceiver

```
public class KafkaReceiver
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide the topic name:");
        String topic = sc.nextLine().trim();

        sc.close();
        KafkaUtils.createTopic(topic); //Attempts to create a topic (might already exist)
        KafkaSubscriber subscriber = KafkaSubscriber.createSubscriber("localhost:9092, kafka:9092",
                                                                    List.of(topic));

        subscriber.start( new RecordProcessor() {
            @Override
            public void onReceive(ConsumerRecord<String, String> r) {
                System.out.println( r.topic() + " , " + r.offset() + " -> " + r.value());
            }
        });
    }
}
```



The start method initializes a thread that will consume events (i.e., Records) from Kafka forever. It receives an instance of the ConsumerRecord interface (defined in-line here) with the code to handle each record (in this example it simply prints the contents of the record).

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (preamble)

```
public class KafkaSubscriber {
    static public KafkaSubscriber createSubscriber( String addr, List<String> topics) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Values for setting the subscription mode (check the documentation on https://kafka.apache.org)
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        // props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");

        props.put(ConsumerConfig.GROUP_ID_CONFIG, UUID.randomUUID().toString());

        // Class that can serialize the key format (string)
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

        // Class that can serialize the value of events (string)
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

        // generates a new Kafka Subscriber with its own Kafka Consumer
        return new KafkaSubscriber( new KafkaConsumer<String, String>(props), topics);
    }

    private static final long POLL_TIMEOUT = 1L;

    final KafkaConsumer<String, String> consumer;

    public KafkaSubscriber(KafkaConsumer<String, String> consumer, List<String> topics) {
        this.consumer = consumer;
        this.consumer.subscribe(topics);
    }
}
```

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (preamble)

```
public class KafkaSubscriber {
    static public KafkaSubscriber createSubscriber( String addr, List<String> topics) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Values for setting the subscription mode (check the documentation on https://kafka.apache.org)
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        // props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");

        props.put(ConsumerConfig.GROUP_ID_CONFIG, UUID.randomUUID().toString());

        // Class that can serialize the key format (string)
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

        // Class that can serialize the value of events (string)
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

        // generates a new Kafka Subscriber with its own Kafka Consumer
        return new KafkaSubscriber( new KafkaConsumer<String, String>(props), topics);
    }

    private static final long POLL_TIMEOUT = 1L;

    final KafkaConsumer<String, String> consumer;

    public KafkaSubscriber(KafkaConsumer<String, String> consumer) {
        this.consumer = consumer;
        this.consumer.subscribe(topics);
    }
}
```

The static method that creates an instance of this class has some parameters that define the behaviour of the KafkaConsumer.

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (preamble)

```
public class KafkaSubscriber {
    static public KafkaSubscriber createSubscriber( String addr, List<String> topics) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Values for setting the subscription mode (check the documentation on https://kafka.apache.org)
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        // props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");

        props.put(ConsumerConfig.GROUP_ID_CONFIG, UUID.randomUUID().toString());

        // Class that can serialize the key format (string)
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

        // Class that can serialize the value of events (string)
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

        // generates a new Kafka Subscriber with its own Kafka Consumer
        return new KafkaSubscriber( new KafkaConsumer<String, String>(props), topics);
    }

    private static final long POLL_TIMEOUT = 1L;

    final KafkaConsumer<String, String> consumer;

    public KafkaSubscriber(KafkaConsumer<String, String>
        this.consumer = consumer;
        this.consumer.subscribe(topics);
    }
}
```

This is the list of Kafka servers as we have seen previously for the Producer

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (preamble)

```
public class KafkaSubscriber {
    static public KafkaSubscriber createSubscriber( String addr, List<String> topics) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Values for setting the subscription mode (check the documentation on https://kafka.apache.org)
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        // props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");

        props.put(ConsumerConfig.GROUP_ID_CONFIG, UUID.randomUUID().toString());

        // Class that can serialize the key format (string)
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

        // Class that can serialize the value of events (string)
        prop

        // g
        retu
    }

    private
    final Ka

    public K
        this
        this
    }
}
```

This parameter defines the behaviour of the consumer in relation to where it starts to consume the stream of records for a topic. Only one value should be used (the two lines here is to show the alternatives) check the documentation to pick the correct one, although the names are quite self-explanatory 😊.

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (preamble)

```
public class KafkaSubscriber {
    static public KafkaSubscriber createSubscriber( String addr, List<String> topics) {
        Properties props = new Properties();

        // List of pairs hostname:port that allows to contact kafka servers
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, addr);

        // Values for setting the subscription mode (check the documentation on https://kafka.apache.org)
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        // props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");

        props.put(ConsumerConfig.GROUP_ID_CONFIG, UUID.randomUUID().toString());

        // Class that can serialize the key format (string)
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

        // Class that can serialize the value of events (string)
```

The GROUP_ID_CONFIG parameter defines the group of the consumer. Consumers in the same group receive different records for each topic (and each record is received by a single member of the group).

In our case we want every consumer to receive all the records, so all of them have to be in distinct groups (that is why we use UUID).

```
        this.consumer.subscribe(topics);
    }
```

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (preamble)

```
public class KafkaSubscriber {
```

```
    static public Kafka  
        Properties prop
```

```
    // List of pai  
    props.put(Cons
```

```
    // Values for  
    props.put(Cons  
    // props.put(C
```

```
    props.put(ConsumerConfig.GROUP_ID_CONFIG, UUID.randomUUID().toString());
```

```
    // Class that can serialize the key format (String)
```

```
    props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
```

```
    // Class that can serialize the value of events (string)
```

```
    props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
```

```
    // generates a new kafka subscriber with its own kafka consumer
```

```
    return new KafkaSubscriber( new KafkaConsumer<String, String>(props), topics);
```

```
}
```

```
private static final long POLL_TIMEOUT = 1L;
```

```
final KafkaConsumer<String, String> consumer;
```

```
public KafkaSubscriber(KafkaConsumer<String, String> consumer, List<String> topics) {
```

```
    this.consumer = consumer;
```

```
    this.consumer.subscribe(topics);
```

```
}
```

We need to indicate how Keys and Values within records should be serialized and deserialized (i.e., their formats). As in the producer we are assuming both the Key and Value to be Strings, hence we use the StringDeserializer for both.

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (preamble)

```
public class KafkaSubscriber {
```

```
    static public KafkaConsumer<String, String> consumer;  
    Properties props;
```

```
    // List of pairs of topic and partition  
    props.put(ConsumerConfig.TOPIC_PARTITION_LIST_CONFIG, "");
```

```
    // Values for consumer  
    props.put(ConsumerConfig.GROUP_ID_CONFIG, UUID.randomUUID().toString());  
    props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
```

We need to indicate how Keys and Values within records should be serialized and deserialized (i.e., their formats). As in the producer we are assuming both the Key and Value to be Strings, hence we use the StringDeserializer for both.

```
    props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
```

```
    // Class that can serialize the key format (String)
```

```
    props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
```

```
    // Class that can serialize the value of events (string)
```

```
    props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
```

```
    // generates a new kafka subscriber with its own kafka consumer
```

```
    return new KafkaSubscriber( new KafkaConsumer<String, String>(props), topics);
```

```
}
```

```
private static final long POLL_TIMEOUT = 1L;
```

```
final KafkaConsumer<String, String> consumer;
```

```
public KafkaSubscriber(KafkaConsumer<String, String> consumer, List<String> topics) {
```

```
    this.consumer = consumer;  
    this.consumer.subscribe(topics);
```

```
}
```

Note that you can always encode a complex java object in text using gson 😊

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (preamble)

```
public class KafkaSubscriber {  
    static public KafkaSubscriber createSubscriber( String addr, List<String> topics) {  
        Properties props = new Properties();
```

```
        // List of pairs hostname:port that allows to contact kafka servers
```

The KafkaConsumer is the class from the Kafka API that is responsible for interacting with the Kafka servers as a consumer. We create it here and will use it to obtain records from Kafka.

Note that you can indicate the list of relevant topics in the Constructor, or use the method subscribe to indicate this list at a later time (there are also methods to unsubscribe from topics).

```
        return new KafkaSubscriber( new KafkaConsumer<String, String>(props), topics);  
    }
```

```
    private static final long POLL_TIMEOUT = 1L;
```

```
    final KafkaConsumer<String, String> consumer;
```

```
    public KafkaSubscriber(KafkaConsumer<String, String> consumer, List<String> topics) {  
        this.consumer = consumer;  
        this.consumer.subscribe(topics);  
    }
```

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (methods to consume records)

```
public void consume(SubscriberListener listener)
    for (;;) {
        ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
        Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
        while(iterator.hasNext()) {
            ConsumerRecord<String, String> r = iterator.next();
            listener.onReceive(r.topic(), r.key(), r.value());
        }
    }

public static interface SubscriberListener {
    void onReceive(String topic, String key, String value);
}

public void start(RecordProcessor recordProcessor) {
    new Thread( () -> {
        for (;;) {
            ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
            Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
            while(iterator.hasNext()) {
                ConsumerRecord<String,String> r = iterator.next();
                recordProcessor.onReceive(r);
            }
        }
    }).start();
}
```

This is the method used in our example. It starts a new thread that will consume records (or events) from the topics subscribed to by this consumer. It receives a class as an argument that implements the RecordProcessor interface that has a single method:

`void onReceive(ConsumerRecord<String, String> r);`

This method is executed for each record received from Kafka with the poll method.

```
public void start(RecordProcessor recordProcessor) {  
    new Thread( () -> {  
        for (;;) {  
            ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));  
            Iterator<ConsumerRecord<String,String>> iterator = records.iterator();  
            while(iterator.hasNext()) {  
                ConsumerRecord<String,String> r = iterator.next();  
                recordProcessor.onReceive(r);  
            }  
        }  
    }).start();  
}
```

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (methods to consume records)

```
public void consume(SubscriberListener listener)
```

```
for (;;) {
```

```
    ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
```

```
    Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
```

```
    while(iterator.hasNext()) {
```

```
        ConsumerRecord<String,String> r = iterator.next();
```

```
        listener.on
```

```
    };
```

```
}
```

```
}
```

```
public static interface
```

```
    void onReceive(Stri
```

```
}
```

```
public void start(Recon
```

```
    new Thread( () -> {
```

```
        for (;;) {
```

```
            ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
```

```
            Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
```

```
            while(iterator.hasNext()) {
```

```
                ConsumerRecord<String,String> r = iterator.next();
```

```
                recordProcessor.onReceive(r);
```

```
            }
```

```
        }
```

```
    }).start();
```

```
}
```

```
}
```

The two methods have very similar bodies; they are basically executing the poll method of the consumer and then processing records received through it.

Lets inspect the code of the example above.

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (methods to consume records)

```
public void consume(SubscriberListener listener)
{
    ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
    Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
    while(iterator.hasNext()) {
        ConsumerRecord<String, String> r = iterator.next();
        listener.onReceive(r.topic(), r.key(), r.value());
    }
}

public static interface SubscriberListener {
    void onReceive(String topic, String key, String value);
}

public void start(RecordProcessor recordProcessor) {
    new Thread( () -> {
        for (;;) {
            ConsumerRecords<String,String> records =
            Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
            while(iterator.hasNext()) {
                ConsumerRecord<String,String> r = iterator.next();
                recordProcessor.onReceive(r);
            }
        }
    }).start();
}
```

The poll method of the KafkaConsumer contacts the Kafka server to obtain a Set of (previously unprocessed) records for the topics that have been previously subscribed to.

This set is encapsulated into a single ConsumerRecords instance.

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (methods to consume records)

```
public void consume(SubscriberListener listener)
```

```
for (;;) {
```

```
    ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
```

```
    Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
```

```
    while(iterator.hasNext()) {
```

```
        ConsumerRecord<String, String> r = iterator.next();
```

```
        listener.onReceive(r.topic(), r.key(), r.value());
```

```
    };
```

```
}
```

```
}
```

```
public static interface SubscriberListener {
```

```
    void onReceive(String topic, String key, String value);
```

```
}
```

```
public static interface KafkaSubscriber {
```

The argument is the time (in milliseconds) for which the operation waits for new records. Upon timeout if no records exist it returns an empty instance of ConsumerRecords.

The method can return immediately if there exist records ready for processing by this consumer.

```
}
```

```
}
```

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (methods to consume records)

```
public void consume(SubscriberListener listener)
    for (;;) {
        ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
        Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
        while(iterator.hasNext()) {
            ConsumerRecord<String,String> r = iterator.next();
            listener.onReceive(r.topic(), r.key(), r.value());
        }
    }
```

```
public static interface SubscriberListener {
    void onReceive(String topic, String key, String value);
}
```

```
public
n
```

To process the records, we get the iterator from the ConsumerRecords and iterate over it.

For each record returned by Kakfa we execute the method onReceive of the class passed as argument to the start method.

```
}
```

KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (methods to consume records)

```
public void consume(SubscriberListener listener)
    for (;;) {
        ConsumerRecords<String, String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
        Iterator<ConsumerRecord<String, String>> iterator = records.iterator();
        while(iterator.hasNext()) {
            ConsumerRecord<String, String> r = iterator.next();
            listener.onReceive(r.topic(), r.key(), r.value());
        }
    }

public static interface SubscriberListener {
    void onReceive(String topic, String key, String value);
}

public void start(RecordProcessor
    new Thread( () -> {
        for (;;) {
            ConsumerRecords<String, String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
            Iterator<ConsumerRecord<String, String>> iterator = records.iterator();
            while(iterator.hasNext()) {
                ConsumerRecord<String, String> r = iterator.next();
                recordProcessor.onReceive(r.topic(), r.key(), r.value());
            }
        }
    }).start();
}
```

Note that records are processed in the same order in which they were received.

So if the topic has a single partition, all consumers (in different groups) will be processing these records in the same order.



KAFKA: PROGRAMMING (SUBSCRIBING AND RECEIVING EVENTS)

Class KafkaSubscriber (methods to consume records)

```
public void consume(SubscriberListener listener)
    for (;;) {
        ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
        Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
        while(iterator.hasNext()) {
            ConsumerRecord<String, String> r = iterator.next();
            listener.onReceive(r.topic(), r.key(), r.value());
        }
    }
```

```
public static interface SubscriberListener {
    void onReceive(String topic, String key, String value);
}
```

```
public void start(RecordProcessor recordProcessor) {
    new Thread( () -> {
        for (;;) {
            ConsumerRecords<String,String> records = consumer.poll(Duration.ofSeconds(POLL_TIMEOUT));
            Iterator<ConsumerRecord<String,String>> iterator = records.iterator();
            while(iterator.hasNext()) {
                ConsumerRecord<String,String> r = iterator.next();
                recordProcessor.onReceive(r.topic(), r.key(), r.value());
            }
        }
    }).start();
}
```

Contrary to the method below, this method does not start a Thread, and it receives a different type of argument: a class that implements the interface SubscriberListener defined below.

KAFKA: PROGRAMMING (EXECUTING EXAMPLES)

You can (and should) test the examples provided in the lab11 code in your machine.

To do so, follow these steps:

1. Start the kafka container as show before in localhost mode
(`sh start-kafka.sh localhost` or
`./start-kafka.sh localhost`)
2. Launch the Receiver and provide a topic (e.g., SD)
3. Launch the Sender and provide a topic and message (the topic must be the same)
4. You can execute the Sender more times to see increasing sequence numbers

KAFKA: PROGRAMMING (EXECUTING EXAMPLES)

Execution of the Sender

```
nmp@Mac lab11 % java -cp target/sd2526-lab11-1.jar lab11.kafka.KafkaSendMsg
Provide the topic name:
SD
Provide the message to send:
Sending message through Kafka!!
log4j:WARN No appenders could be found for logger (org.apache.kafka.common.config.AbstractConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Topic: SD already exists...
java.util.concurrent.ExecutionException: org.apache.kafka.common.errors.TopicExistsException: Topic 'SD' already exists.
    at java.base/java.util.concurrent.CompletableFuture.wrapInExecutionException(CompletableFuture.java:345)
    at java.base/java.util.concurrent.CompletableFuture.reportGet(CompletableFuture.java:440)
    at java.base/java.util.concurrent.CompletableFuture.get(CompletableFuture.java:2094)
    at org.apache.kafka.common.internals.KafkaFutureImpl.get(KafkaFutureImpl.java:155)
    at lab11.kafka.KafkaUtils.createTopic(KafkaUtils.java:34)
    at lab11.kafka.KafkaUtils.createTopic(KafkaUtils.java:22)
    at lab11.kafka.KafkaSendMsg.main(KafkaSendMsg.java:18)
Caused by: org.apache.kafka.common.errors.TopicExistsException: Topic 'SD' already exists.
Published to topic SD with offset 0
Message published with offset (sequence number): 0
```

Execution of the Receiver (in Eclipse)

```
nmp@Mac lab11 % java -cp target/sd2526-lab11-1.jar lab11.kafka.KafkaReceiver
Provide the topic name:
SD
log4j:WARN No appenders could be found for logger (org.apache.kafka.common.config.AbstractConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Topic SD was created successfully
SD , 0 -> Sending message through Kafka!!
```

KAFKA: PROGRAMMING (EXECUTING EXAMPLES)

Execution of the Sender

```
nmp@Mac lab11 % java -cp target/sd2526-lab11-1.jar lab11.kafka.KafkaSendMsg
```

```
Provide the topic name:
```

```
SD
```

```
Provide the message to send:
```

```
Sending message through Kafka!!
```

```
log4j:WARN No appenders could be found for logger (org.apache.kafka.common.config.AbstractConfig).
```

```
log4j:WARN Please initialize the log4j system properly.
```

```
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
```

```
Topic: SD already exists...
```

```
java.util.concurrent.ExecutionException: org.apache.kafka.common.errors.TopicExistsException: Topic 'SD' already exists.
```

```
    at java.base/java.util.concurrent.CompletableFuture.wrapInExecutionException(CompletableFuture.java:345)
```

```
    at java.base/java.util.concurrent.CompletableFuture.reportGet(CompletableFuture.java:440)
```

```
    at java.base/java.util.concurrent.CompletableFuture.get(CompletableFuture.java:1960)
```

```
    at org.apache.kafka.common.internals.Topic.createTopic(Topic.java:100)
```

```
    at lab11.kafka.KafkaUtils.createTopic(KafkaUtils.java:100)
```

```
    at lab11.kafka.KafkaUtils.createTopic(KafkaUtils.java:100)
```

```
    at lab11.kafka.KafkaSendMsg.sendMessage(KafkaSendMsg.java:100)
```

```
Caused by: org.apache.kafka.common.errors.TopicExistsException: Topic 'SD' already exists.
```

```
Published to topic SD with offset 0
```

```
Message published with offset (sequence number) 0
```

The exception in the sender its because the Topic SD already existed (created by the receiver)

Execution of the Receiver (in Eclipse)

```
nmp@Mac lab11 % java -cp target/sd2526-lab11-1.jar lab11.kafka.KafkaReceiver
```

```
Provide the topic name:
```

```
SD
```

```
log4j:WARN No appenders could be found for logger (org.apache.kafka.common.config.AbstractConfig).
```

```
log4j:WARN Please initialize the log4j system properly.
```

```
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
```

```
Topic SD was created successfully
```

```
SD , 0 -> Sending message through Kafka!!
```

KAFKA: PROGRAMMING (EXECUTING EXAMPLES)

Execution of the Sender

```
nmp@Mac lab11 % java -cp target/sd2526-lab11-1.jar lab11.kafka.KafkaSendMsg
Provide the topic name:
SD
Provide the message to send:
Sending message through Kafka!!
log4j:WARN No appenders could be found for logger (org.apache.kafka.common.config.AbstractConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Topic: SD already exists...
java.util.concurrent.ExecutionException: org.apache.kafka.common.errors.TopicExistsException: Topic 'SD' already exists.
    at java.base/java.util.concurrent.CompletableFuture.wrapInExecutionException(CompletableFuture.java:345)
    at java.base/java.util.concurrent.CompletableFuture.reportGet(CompletableFuture.java:440)
    at java.base/java.util.concurrent.CompletableFuture.get(CompletableFuture.java:2094)
    at org.apache.kafka.common.internals.KafkaFutureImpl.get(KafkaFutureImpl.java:155)
    at lab11.kafka.KafkaUtils.createTopic(KafkaUtils.java:34)
    at lab11.kafka.KafkaUtils.createTopic(KafkaUtils.java:22)
    at lab11.kafka.KafkaSendMsg.main(KafkaSendMsg.java:18)
Caused by: org.apache.kafka.common.errors.TopicExistsException: Topic 'SD' already exists.
Published to topic SD with offset 0
Message published with offset (sequence number): 0
```

Execution of the Receiver (in Eclipse)

```
nmp@Mac lab11 % java -cp target/sd2526-lab11-1.jar lab11.kafka.KafkaReceiver
Provide the topic name:
SD
log4j:WARN No appenders could be found for logger (org.apache.kafka.common.config.AbstractConfig).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.
Topic SD was created successfully
SD , 0 -> Sending message through Kafka!!
```

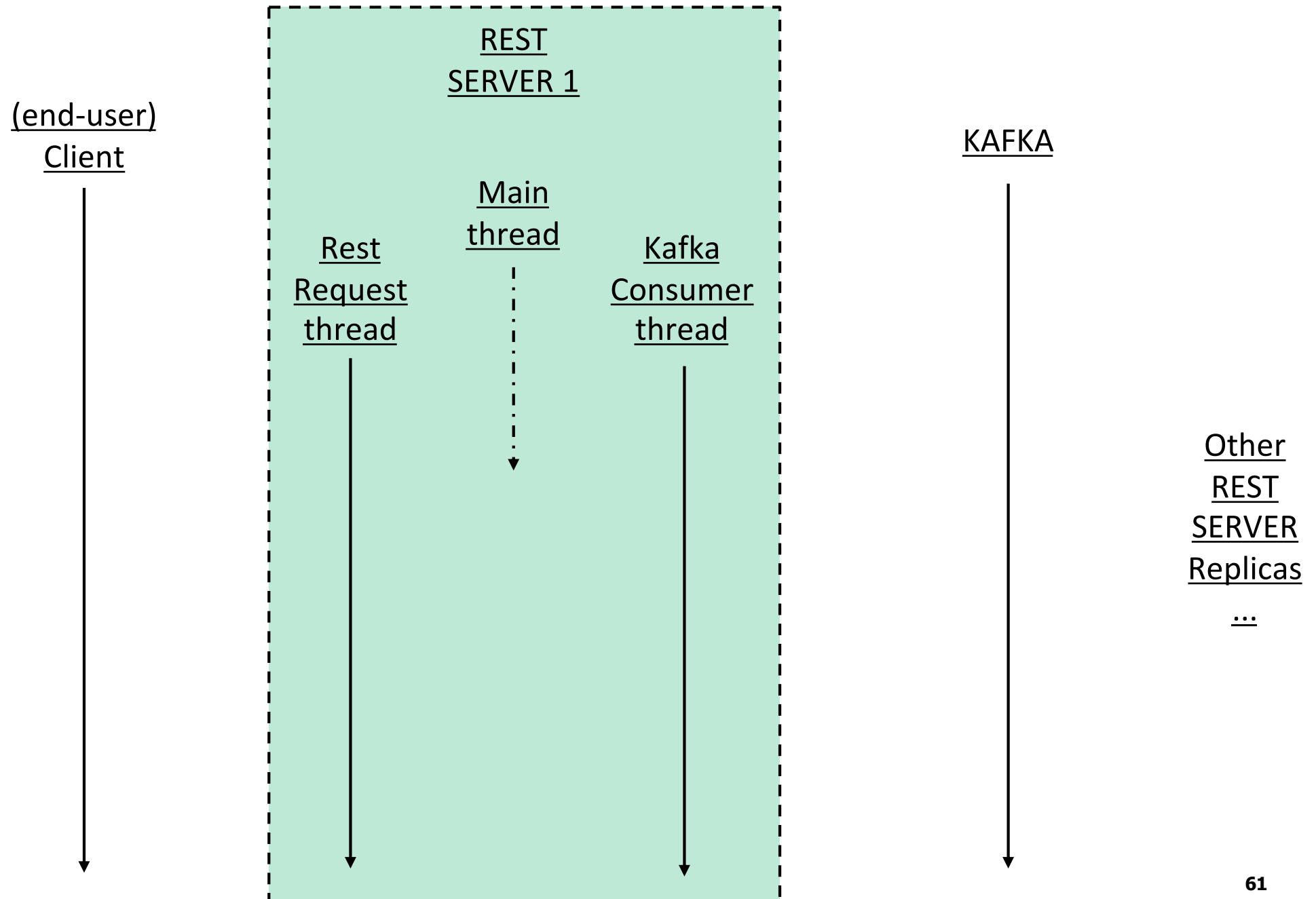
The receiver indeed receives the message with offset zero:

GOALS

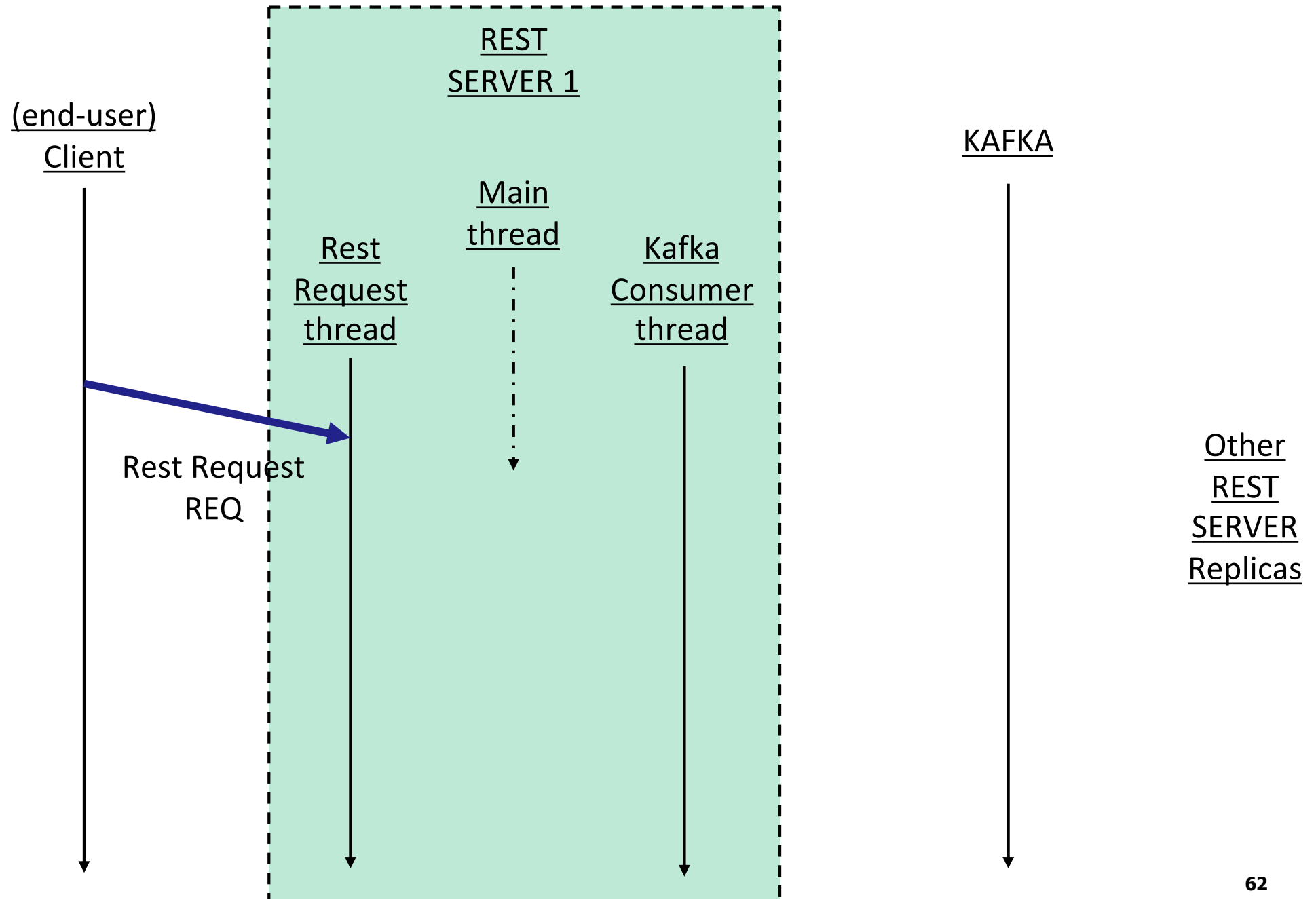
In the end of this lab, you should be able to:

- **In relation to Kafka**
 - **Understand the abstractions provided by Kafka**
 - Understand how to publish events in Java
 - Understand how to subscribe event in Java
 - **Understand the asynchronous processing of Kafka received operations in the context of replication in the project.**
- Zookeeper
 - Understand the abstractions provided by Zookeeper
 - Understand how to create a tuple (znode) in Java
 - Understand how to monitor a tuple (znode) in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- Overall Revision of the Project Goals

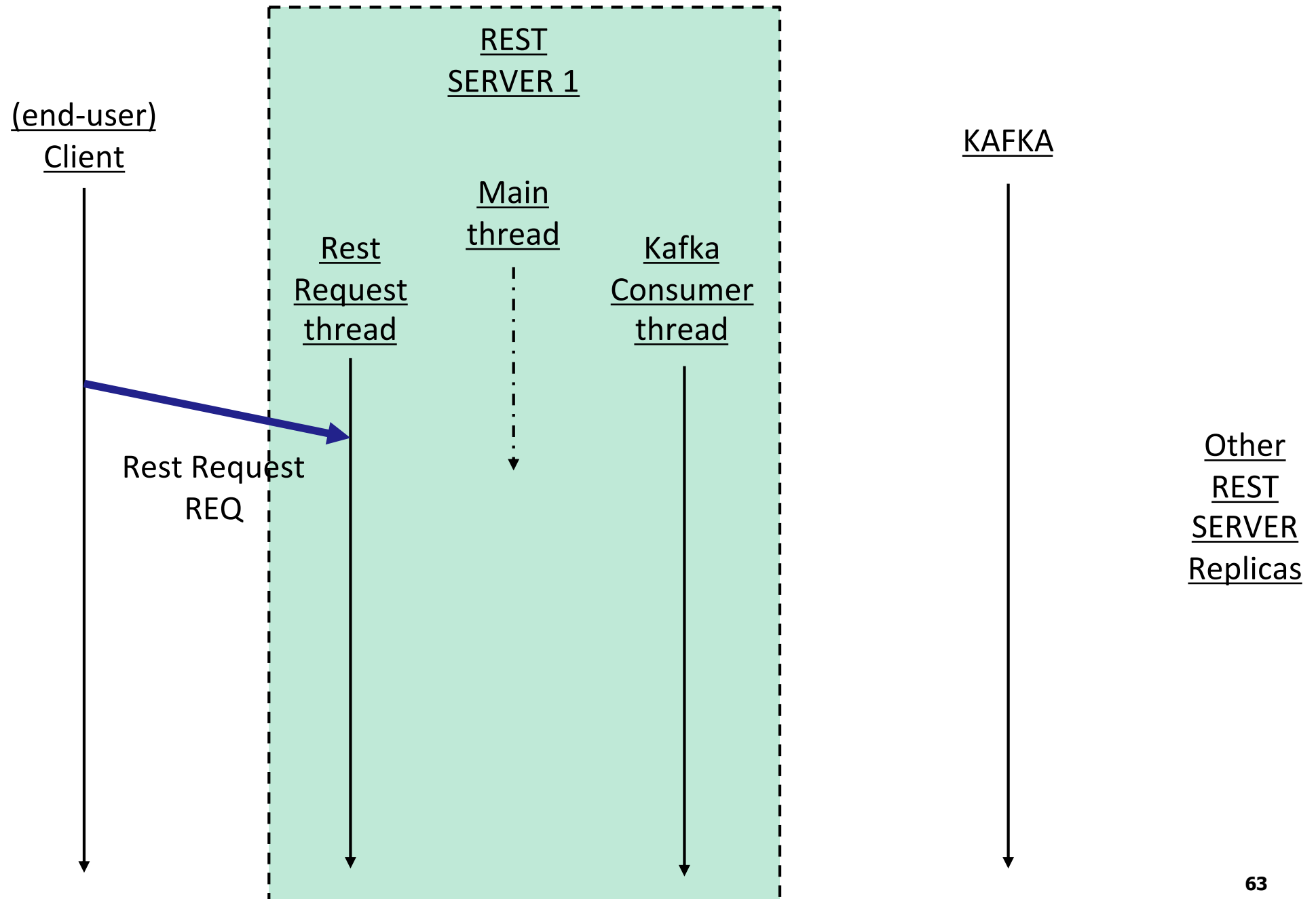
KAFKA: ASYNCHRONY AND REST SERVERS



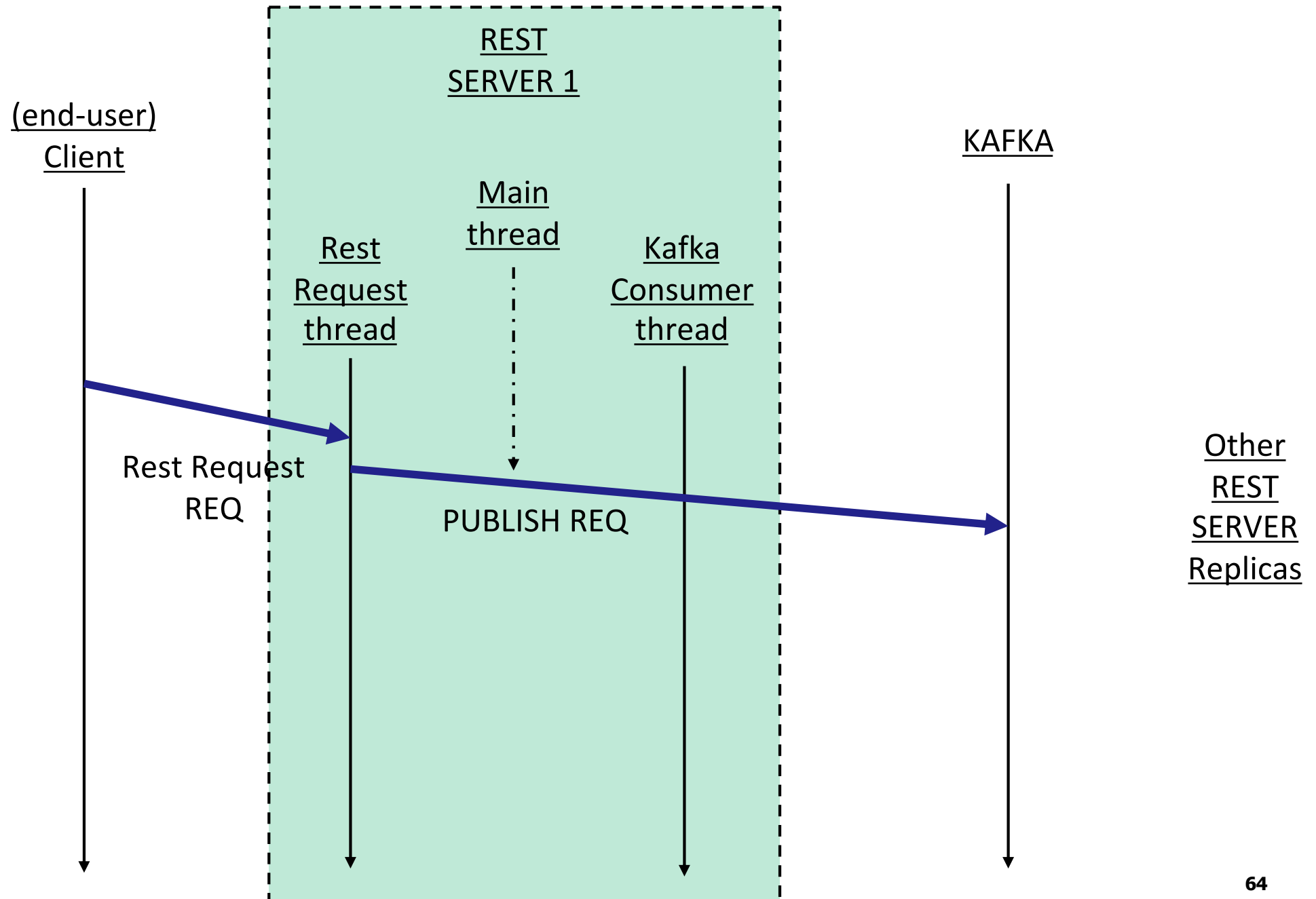
KAFKA: ASYNCHRONY AND REST SERVERS



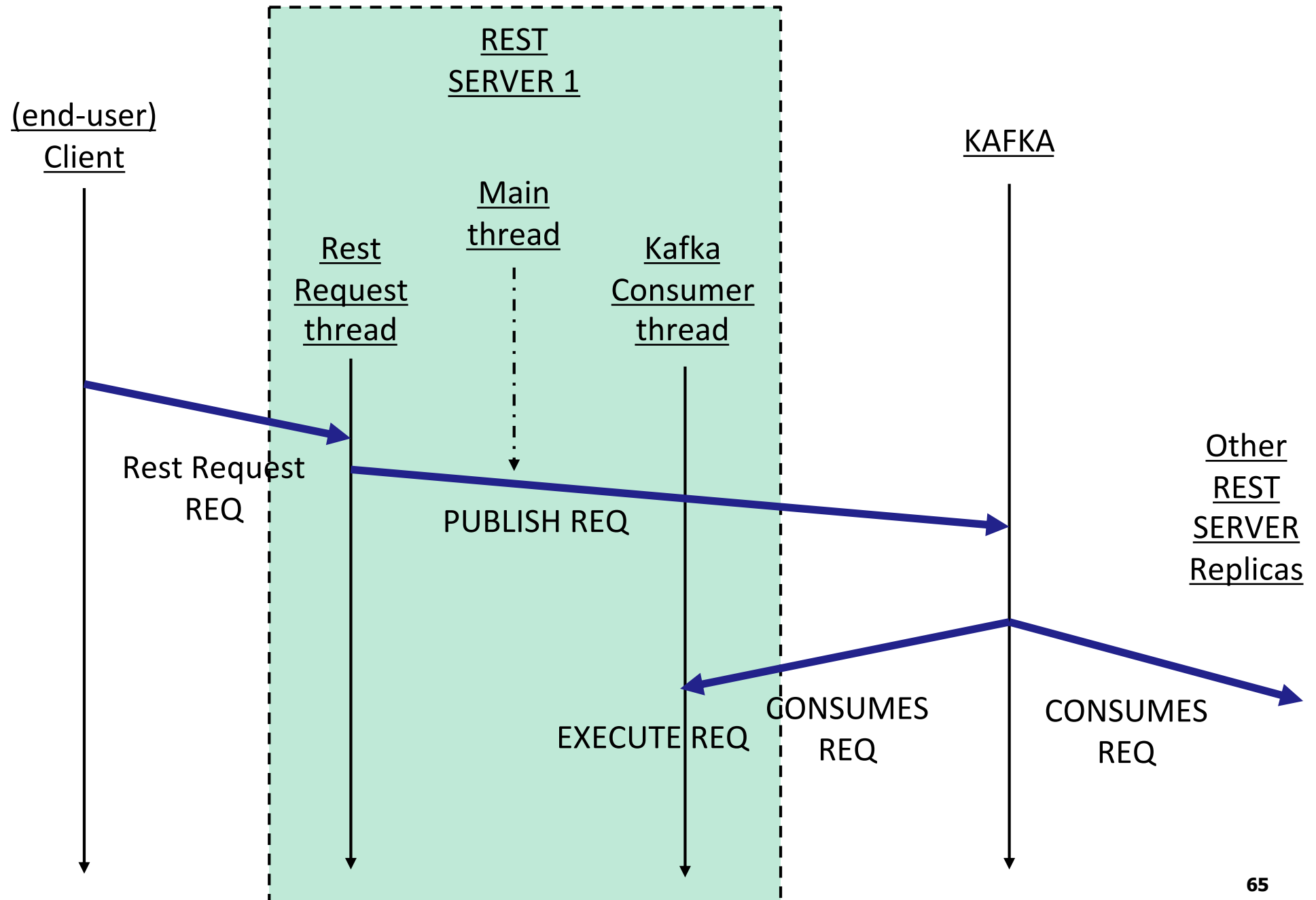
KAFKA: ASYNCHRONY AND REST SERVERS



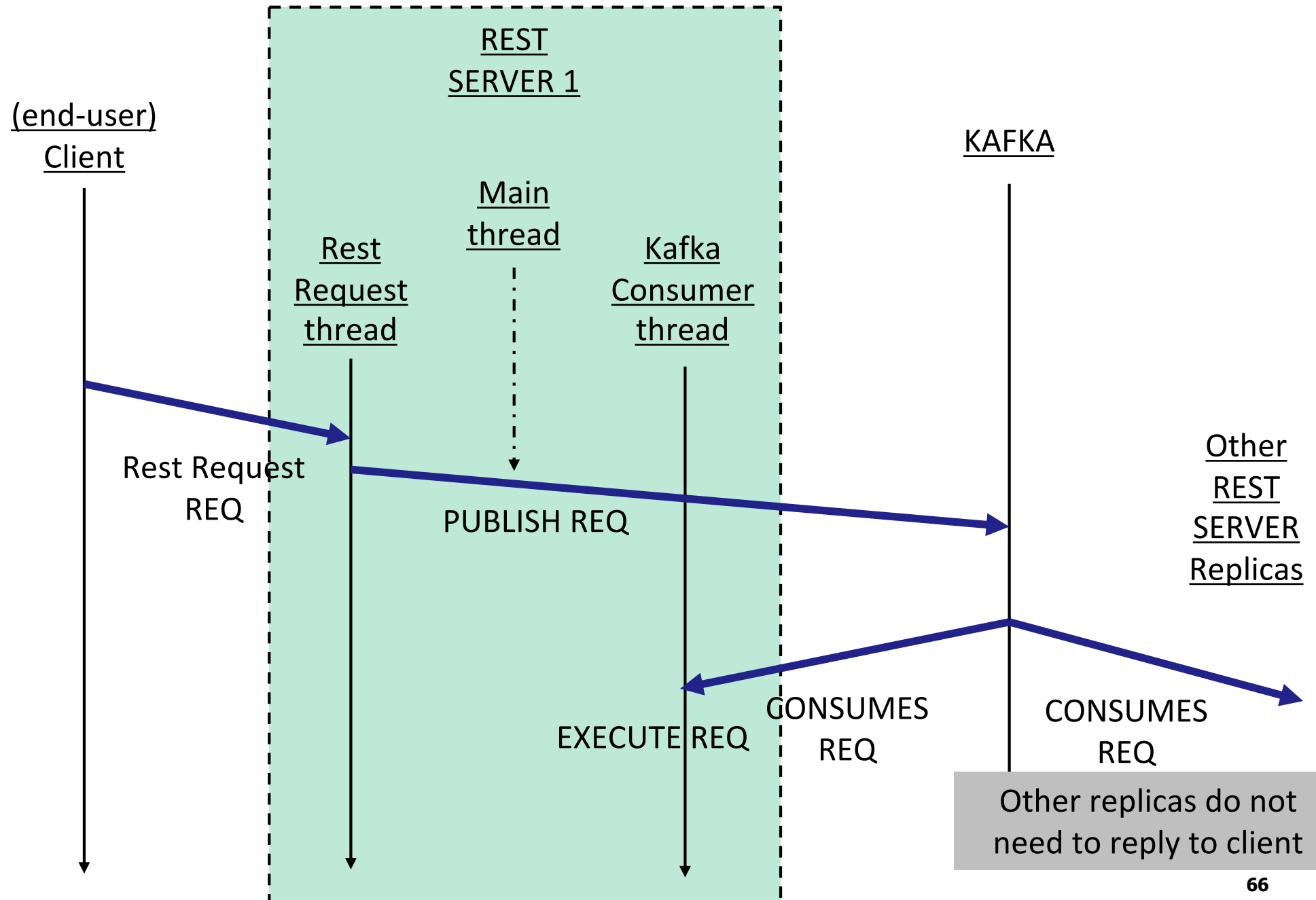
KAFKA: ASYNCHRONY AND REST SERVERS



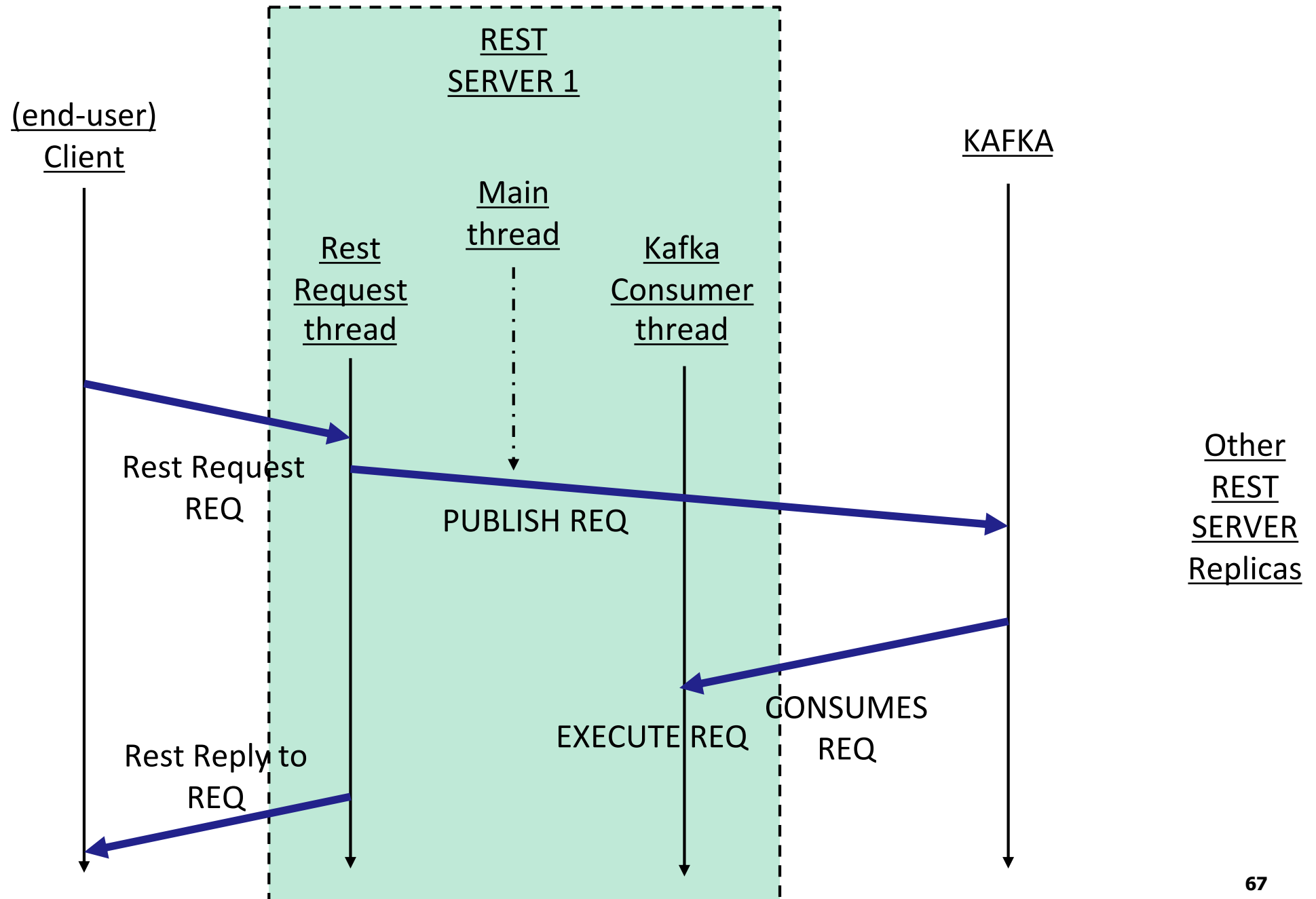
KAFKA: ASYNCHRONY AND REST SERVERS



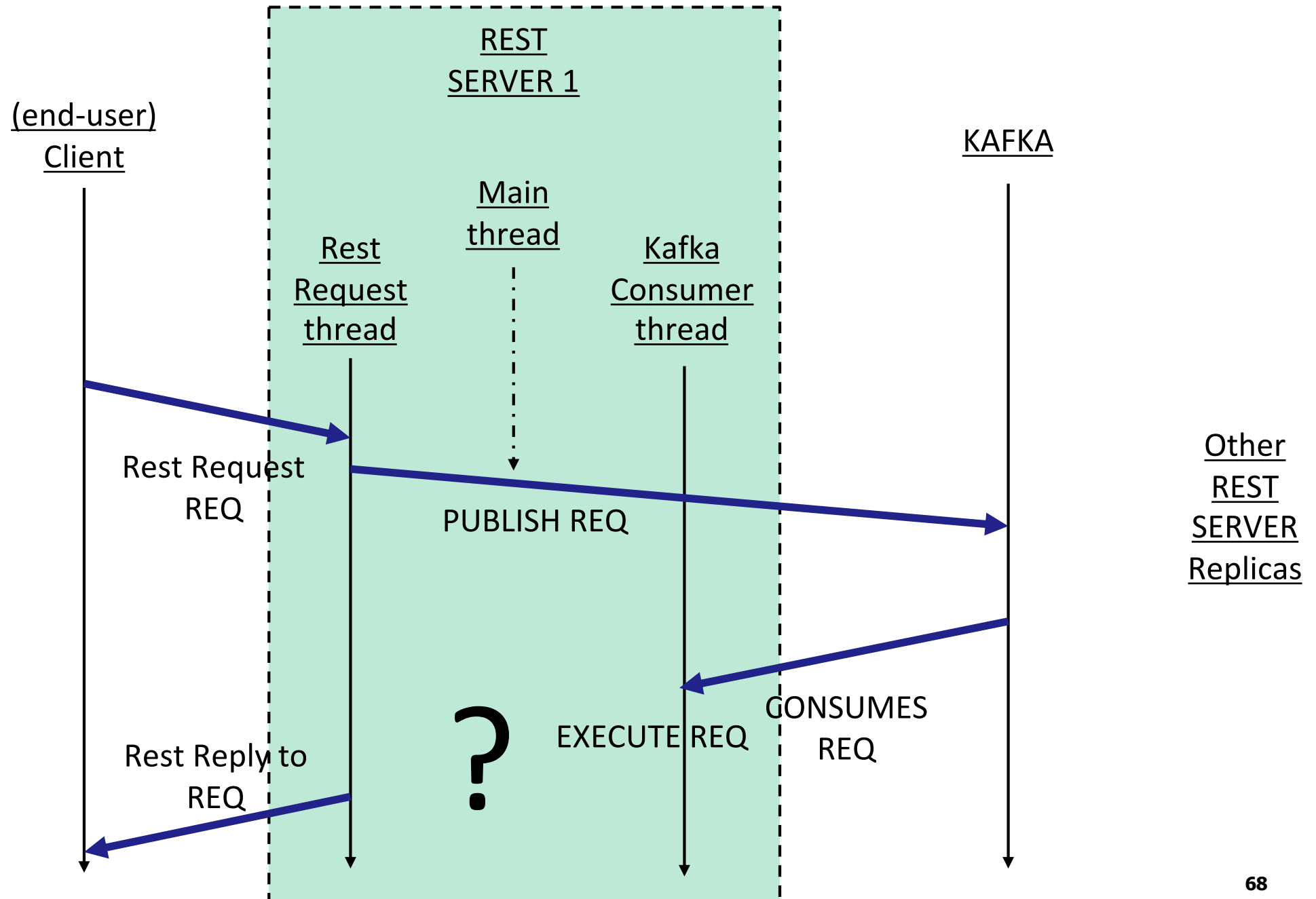
KAFKA: ASYNCHRONY AND REST SERVERS



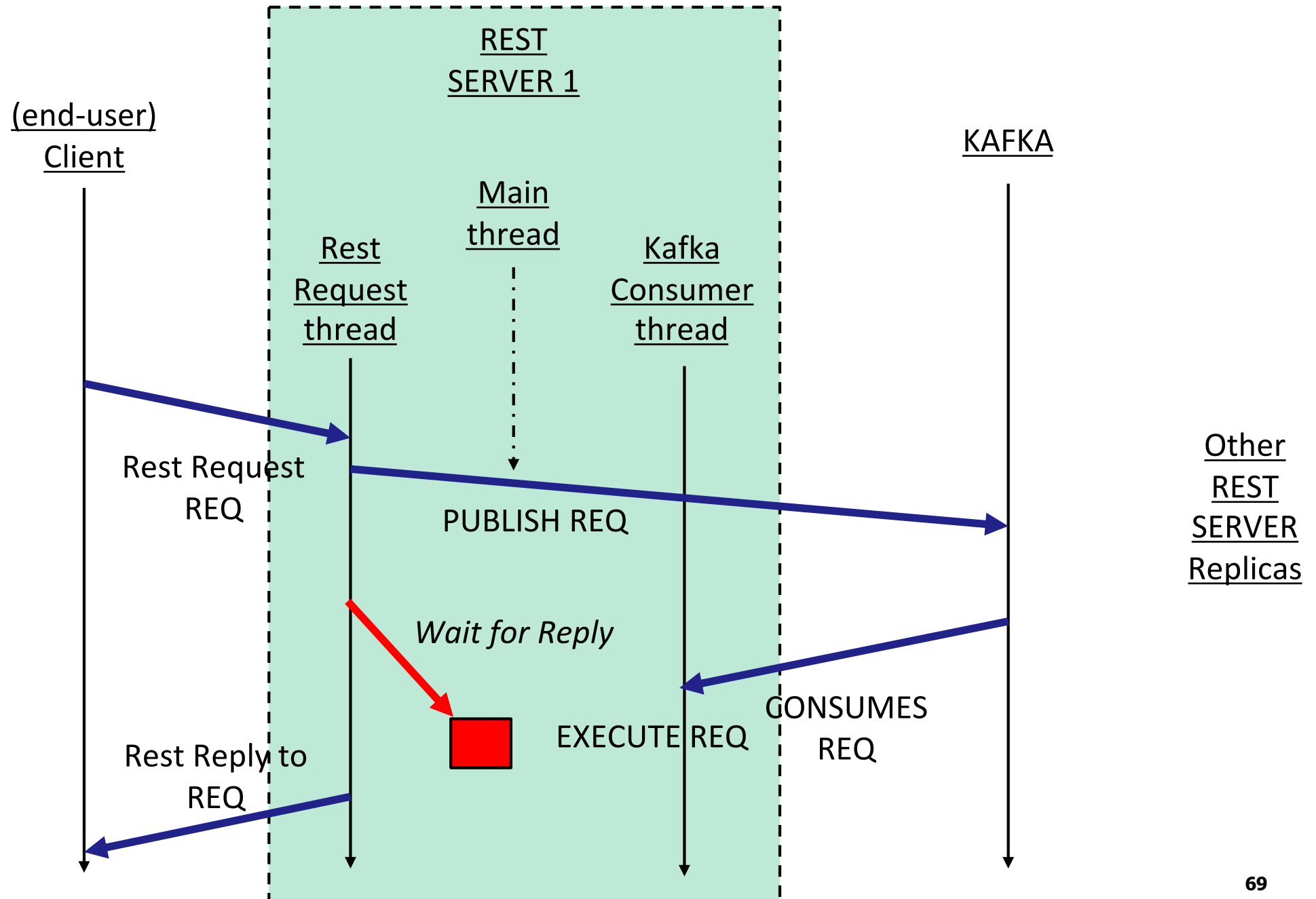
KAFKA: ASYNCHRONY AND REST SERVERS



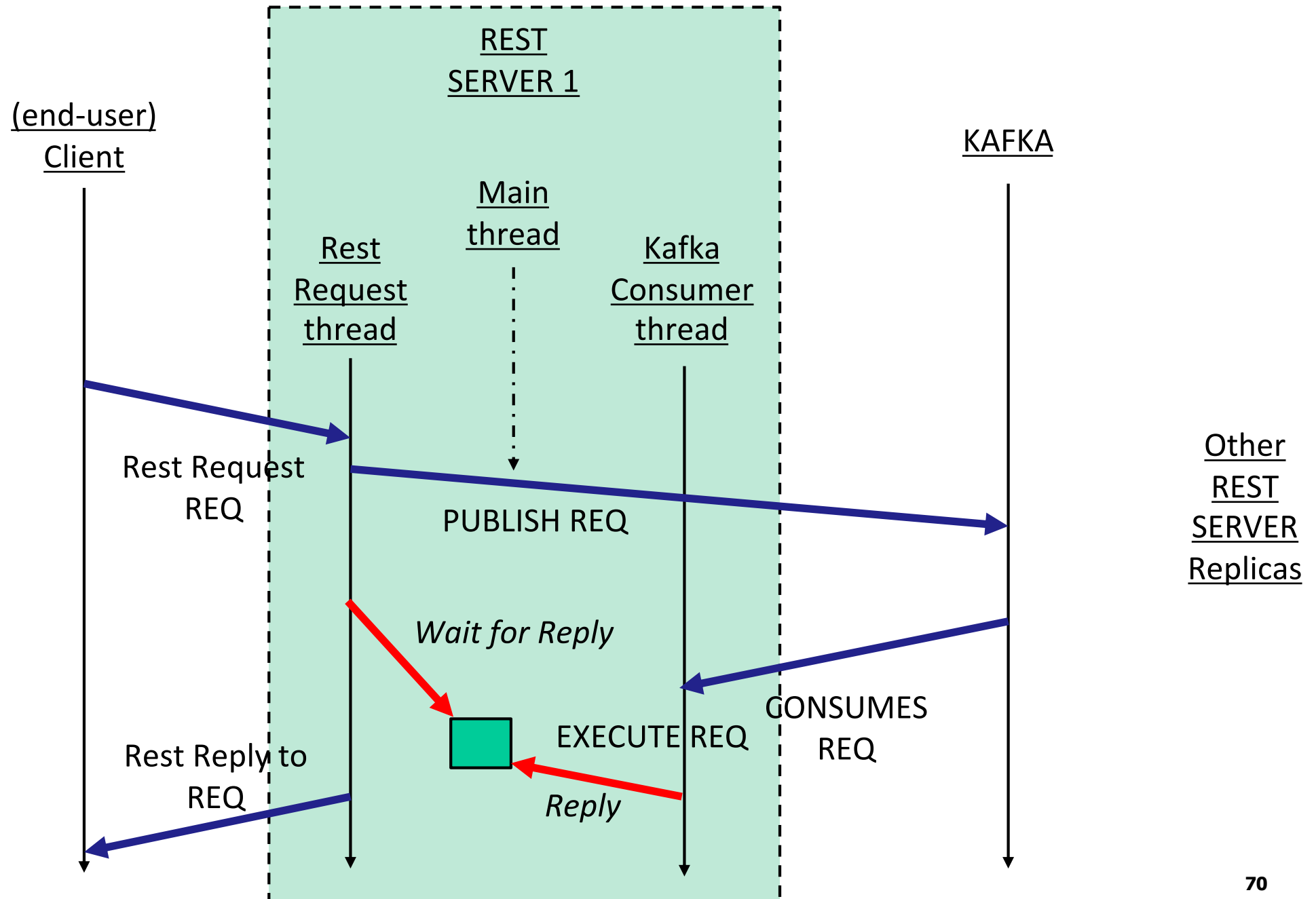
KAFKA: ASYNCHRONY AND REST SERVERS



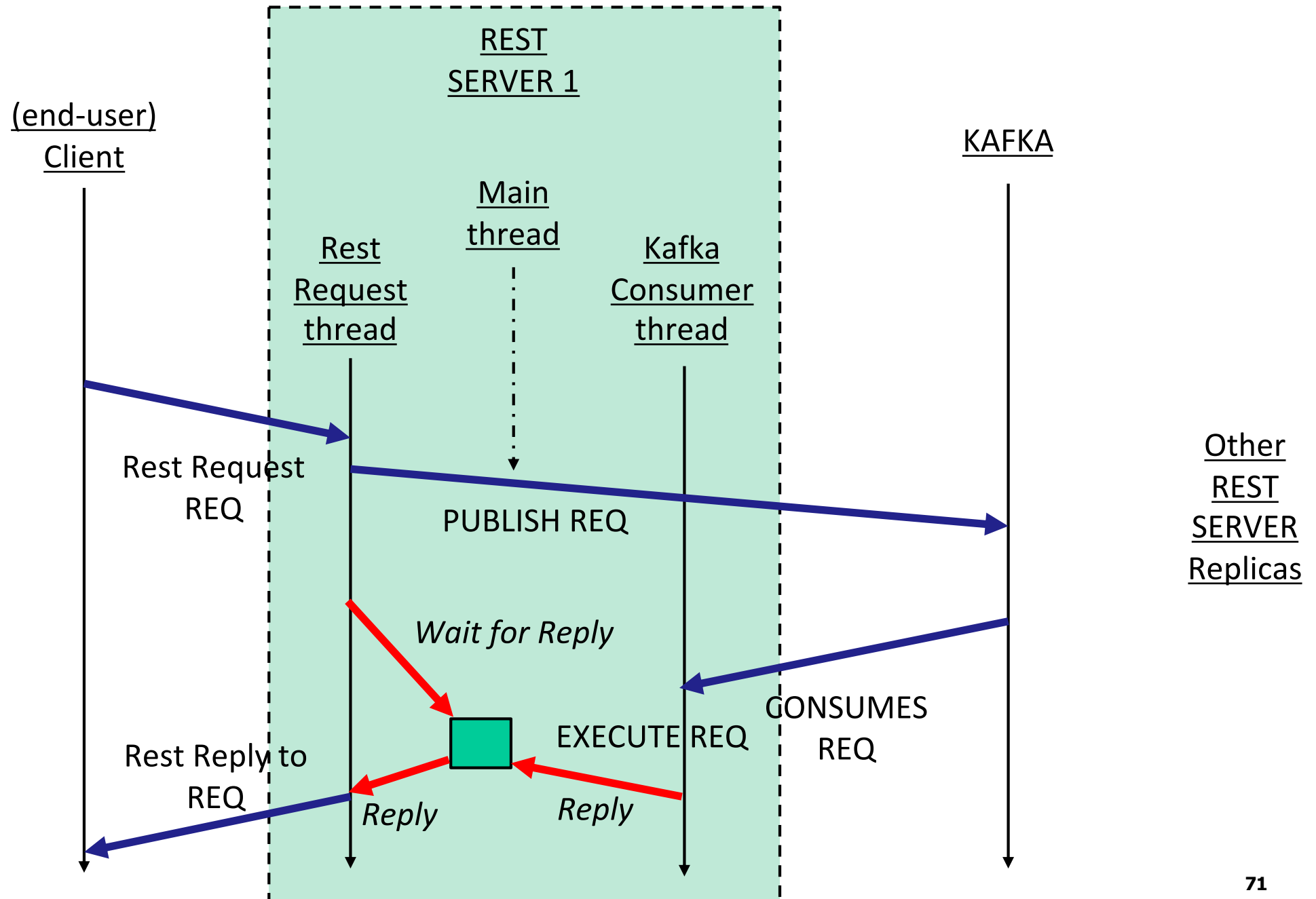
KAFKA: ASYNCHRONY AND REST SERVERS



KAFKA: ASYNCHRONY AND REST SERVERS



KAFKA: ASYNCHRONY AND REST SERVERS



KAFKA: ASYNCHRONY AND REST SERVERS

To simplify this interaction, we provide in this lab's code a **SyncPoint class** that can be used by the thread that receives the client REST request to block until the Kafka consumer thread delivers the operation and the result of executing the operation becomes available.

This class also offers mechanisms that block a thread until a given sequence number (offset) has been processed by the local server, which allows us to ensure that clients do not see state older than they have previously seen.

Like the Discovery, this class will have to be accessible both to the REST Resources class and the Kafka Consumer class (hence it has the singleton pattern defined 😊 correctly defined contrary to some of your Discovery implementations 😊).

KAFKA: ASYNCHRONY AND REST SERVERS

Class SyncPoint (State and Creation)

```
public class SyncPoint {
    private final ConcurrentHashMap<Long, String> result;
    private long version;

    private SyncPoint() {
        this.result = new ConcurrentHashMap<Long, String>();
        this.version = 0;
    }

    private static SyncPoint instance = null;

    public static synchronized SyncPoint getSyncPoint() {
        if(SyncPoint.instance == null)
            SyncPoint.instance = new SyncPoint();
        return SyncPoint.instance;
    }
}
```

KAFKA: ASYNCHR

The class uses the singleton pattern, by having a private constructor, a static variable of this type, which is always returned (if previously instantiated) by the static method.

```
public class SyncPoint {  
    private final ConcurrentHashMap<Long, String> result;  
    private long version;
```

```
    private SyncPoint() {  
        this.result = new ConcurrentHashMap<Long, String>();  
        this.version = 0;  
    }  
  
    private static SyncPoint instance = null;  
  
    public static synchronized SyncPoint getSyncPoint() {  
        if(SyncPoint.instance == null)  
            SyncPoint.instance = new SyncPoint();  
        return SyncPoint.instance;  
    }  
}
```

KAFKA: ASYNCHRONY AND REST SERVERS

Class SyncPoint (State and Creation)

```
private final ConcurrentHashMap<Long, String> result;  
private long version;
```

```
private SyncPoint() {  
    this.result = new ConcurrentHashMap<Long, String>();  
    this.version = 0;  
}
```

```
private static SyncPoint instance = null;
```

```
public static SyncPoint getInstance() {  
    if(SyncPoint.instance == null) {  
        SyncPoint instance = new SyncPoint();  
        return instance;  
    }  
}
```

The class is quite simple, it has a long version that will represent the offset of the last operation registered in it.

It has a concurrent hash map that stores the results of operations executed after their order has been defined.

KAFKA: ASYNCHRONY AND REST SERVERS

Class SyncPoint (relevant methods)

```
public synchronized String waitForResult( long n) {
    while( version < n) {
        try {
            wait();
        } catch (InterruptedException e) {
            // nothing to be done here
        }
    }
    return result.remove(n);
}
public synchronized void waitForVersion( long n) {
    while( version < n) {
        try {
            wait();
        } catch (InterruptedException e) {
            // nothing to be done here
        }
    }
}
public synchronized void setResult( long n, String res) {
    if( n < version)
        throw new RuntimeException("Version " + n + " is already set");
    if ( res != null) {
        result.put(n, res);
    }
    version = n;
    notifyAll();
}
}
```

KAFKA: ASYNCHRONY AND REST SERVERS

Class SyncPoint (relevant methods)

```
public synchronized String waitForResult( long n) {  
    while( version <  
        try {  
            wait();  
        } catch (Inte  
            // nothin  
        }  
    }  
    return result.rem  
}  
public synchronized v  
    while( version <  
        try {  
            wait();  
        } catch (Inte  
            // nothing to be done here  
        }  
    }  
}
```

- The method setResult allows us to do two things:
- 1) Register the last offset (sequence number) associated with a request previously propagated through Kafka.
 - 2) Store associated with that number a response associated with the processing of that request.

```
public synchronized void setResult( long n, String res) {  
    if( n < version)  
        throw new RuntimeException("Version " + n + " is already set");  
    if ( res != null) {  
        result.put(n, res);  
    }  
    version = n;  
    notifyAll();  
}
```

KAFKA: ASYNCHRONY AND REST SERVERS

Class SyncPoint (relevant methods)

```
public synchronized String waitForResult( long n) {
```

```
    while( version < n) {
        try {
            wait();
        } catch (InterruptedException e) {
            // nothing to do here
        }
    }
```

```
    return result.remove(n);
}
```

```
public synchronized void setResult( long n, String res) {
```

```
    while( version < n) {
        try {
            wait();
        } catch (InterruptedException e) {
            // nothing to be done here
        }
    }
```

```
    if ( n < version)
        throw new RuntimeException("Version " + n + " is already registered");
    if ( res != null) {
        result.put(n, res);
    }
    version = n;
    notifyAll();
}
```

```
    public synchronized void setResult( long n, String res) {
        if( n < version)
            throw new RuntimeException("Version " + n + " is already registered");
        if ( res != null) {
            result.put(n, res);
        }
        version = n;
        notifyAll();
    }
}
```

- The method setResult allows us to do two things:
- 1) Register the last offset (sequence number) associated with a request previously propagated through Kafka.
 - 2) Store associated with that number a response associated with the processing of that request.

Note that this API expects operation results to be registered in the order of the offset of operations.

KAFKA: ASYNCHRONY AND REST SERVERS

Class SyncPoint (relevant methods)

```
public synchronized String waitForResult( long n) {  
    while( version < n) {  
        try {  
            wait();  
        } catch (InterruptedException e) {  
            // nothing to be done here  
        }  
    }  
    return result.remove(n);  
}
```

```
public synchronized void waitForVersion( long n) {  
    while( version < n) {  
        try {  
            wait();  
        } catch (InterruptedException e) {  
            // nothing to be done here  
        }  
    }  
}  
  
public synchronized void setResult( long n, String res) {  
    if( n < version)  
        throw new RuntimeException("Version " + n + " is out of range")  
    if ( res != null) {  
        result.put(n, res);  
    }  
    version = n;  
    notifyAll();  
}
```

This method should be used by the thread that received the request from the client to wait for the operation to be ordered and delivered by kafka (and executed by the Consumer) to obtain the reply that should be sent to the client.

The argument is the offset of the record sent to Kafka.

KAFKA: ASYNCHRONY AND REST SERVERS

Class SyncPoint (relevant methods)

```
public synchronized String waitForResult( long n) {  
    while( version < n) {  
        try {  
            wait();  
        } catch (InterruptedException e) {  
            // nothing to be done here  
        }  
    }  
    return result.remove(n);  
}
```

```
public synchronized void waitForVersion( long n) {  
    while( version < n) {  
        try {  
            wait();  
        } catch (InterruptedException e) {  
            // nothing to be done here  
        }  
    }  
}
```

```
public synchronized void setResult( long n, String res) {  
    if( n < version)  
        throw new RuntimeException("Version " + n + " is out of order");  
    if ( res != null) {  
        result.put(n, res);  
    }  
    version = n;  
    notifyAll();  
}
```

Blocks the caller until the local Kafka consumer has processed and applied all operations up to the specified offset.

This can be used in a read operation to ensure that the local state reflects all previously acknowledged writes (Causal Consistency).

GOALS

By the end of this lab, you should be able to:

- In relation to Kafka
 - Understand the abstractions provided by Kafka
 - Understand how to publish events in Java
 - Understand how to subscribe event in Java
 - Understand the asynchronous processing of Kafka received operations in the context of replication in the project.
- **Zookeeper**
 - **Understand the abstractions provided by Zookeeper**
 - Understand how to create a tuple (znode) in Java
 - Understand how to monitor a tuple (znode) in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- Understand the implementation path for the different implementation alternatives for Replication in the second project.

ZOOKEEPER: DISTRIBUTED TUPLE SPACE

Apache Zookeeper is a (centralized) popular solution to provide powerful abstractions for distributed applications including:

- Naming schemes (i.e., attribute a unique identifier to a process).
- Distributed Synchronization.
- Distributed Consensus/Leader election.
- Management of Configuration information.

ZOOKEEPER: DISTRIBUTED TUPLE SPACE

Apache Zookeeper is a (centralized) popular solution to provide powerful abstractions for distributed applications including:

- Naming schemes (i.e., attribute a unique identifier to a process).
- Distributed Synchronization.
- **Distributed Consensus/Leader election.**
- Management of Configuration information.

ZOOKEEPER: DISTRIBUTED TUPLE SPACE

Apache Zookeeper is a popular system that provides an API to store control information in a reliable and strongly consistent way in a hierarchical tuple space.

Some relevant aspects of zookeeper:

- Tuples have a unique name and optionally a value associated. A tuple is named a znode in zookeeper nomenclature.
- Znodes are hierarchical like the directory structure of a file system. You can have a zone named 'zoo'; a child zone named 'one' has the name '/zoo/one'
- You can watch for modifications in a znode (and all its children) and be notified by zookeeper whenever a change happens.

ZOOKEEPER: DISTRIBUTED TUPLE SPACE

A znode can be created using two different modes:

- **Persistent:** the znode will continue to exist even if the process (i.e., client) that created it fails or disconnects.
- **Ephemeral:** the znode will be automatically deleted when the session/connection of the process (i.e., client) that created it terminates or fails.
- Additionally, a znode can be created with a **Sequential** option that appends a unique sequential node (per parent znode) number to the name of the znode being created.

ZOOKEEPER: DISTRIBUTED TUPLE SPACE

In the context of the project, one of the variants of replication that you can build is primary/secondary, where there is a leader that is responsible for executing operations, having all replicas act as a backup to that primary.

However, when the primary fails, one of the secondary replicas must take its place and become the new primary. Evidently only one of the replicas can be the primary at each moment.

This is an instance of the “leader election” problem, and zookeeper can be used to address this problem in practice (for most fault scenarios).

Check the lectures for details.

ZOOKEEPER: PROGRAMMING (PRELIMINARIES)

The example for this week lab (in clip as lab11.zip) also includes several examples of Zookeeper usage, including the creation of a znode, reading the children of a znode, and also a simplistic leader election example.

ZOOKEEPER: PROGRAMMING (PRELIMINARIES)

You need to add maven dependencies to use Zookeeper in your project (file pom.xml):

The main dependency is:

```
<dependency>
  <groupId>org.apache.zookeeper</groupId>
  <artifactId>zookeeper</artifactId>
  <version>3.9.5</version>
</dependency>
```

These additional dependencies avoid some errors reported by zookeeper clients:

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>2.0.17</version>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-log4j12</artifactId>
  <version>2.0.17</version>
</dependency>
```

ZOOKEEPER: PROGRAMMING (PRELIMINARIES)

To use and interact with Zookeeper its server must be running.

In the support project for this lab you have a script file that executes a docker container with Zookeeper ready for use in either your local machine (address localhost) and in the context of the SD project tester (under the name kafka): **start-kafka.sh**

Note that the script is the same as for Kafka, since Kafka also requires Zookeeper to run, we have both systems executing in the same container (the port to contact Zookeeper however is different).

GOALS

By the end of this lab, you should be able to:

- In relation to Kafka
 - Understand the abstractions provided by Kafka
 - Understand how to publish events in Java
 - Understand how to subscribe event in Java
 - Understand the asynchronous processing of Kafka received operations in the context of replication in the project.
- **Zookeeper**
 - Understand the abstractions provided by Zookeeper
 - **Understand how to create a tuple (znode) in Java**
 - Understand how to monitor a tuple (znode) in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- Overall Revision of the Project Goals

KAFKA: PROGRAMMING (CREATE A TUPLE)

To show how you can create Tuples (i.e., a znode) on Zookeeper, we will use a simple application (main) that serves only as an example.

In your project your own servers are going to be responsible for creating and reading znodes from Zookeeper and there is no need for new classes with main for this purpose.

The Javadoc of Zookeeper (for the latest version 3.9.5) can be found in:

<https://www.javadoc.io/doc/org.apache.zookeeper/zookeeper/latest/index.html>

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperWriter (Example with Main)

```
public class ZookeeperWriter {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with /) :");  
        String path = sc.nextLine().trim();  
  
        System.out.println("Provide a value :");  
        String value = sc.nextLine().trim();  
        sc.close();  
  
        ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");  
        zk.write(path, null, CreateMode.PERSISTENT);  
  
        String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);  
        System.out.println("create new znode: " + newPath);  
    }  
}
```

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperWriter (Example with Main)

```
public class ZookeeperWriter {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Except  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with  
        String path = sc.nextLine().trim();  
  
        System.out.println("Provide a value :");  
        String value = sc.nextLine().trim();  
        sc.close();  
  
        ZookeeperProcessor zk = new ZookeeperProcessor( "local  
        zk.write(path, null, CreateMode.PERSISTENT);  
  
        String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);  
        System.out.println("create new znode: " + newPath);  
    }  
}
```

The code creates a persistent znode (without a value associated) and a child znode that is ephemeral to which it associates a value.

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperWriter (Example with Main)

```
public class ZookeeperWriter {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with /) :");  
        String path = sc.nextLine().trim();  
  
        System.out.println("Provide a value :");  
        String value = sc.nextLine().trim();  
        sc.close();  
  
        ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");  
        zk.write(path, null, CreateMode.PERSISTENT);  
  
        String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL);  
        System.out.println("create new znode: " + newPath);  
    }  
}
```



This is simple code for asking the user to provide the name of the znode that it will create and the value to store in the internal znode that will be created.

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

The ZookeeperProcessor class is an auxiliary class that we will use to interact with Zookeeper (we will see the code in a bit).

The argument, similar to kafka, is a string containing the location of zookeeper servers (pairs hostname:port separated by commas).

```
System.out.println("Provide a value :");  
String value = sc.nextLine().trim();  
sc.close();
```

```
ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");  
zk.write(path, null, CreateMode.PERSISTENT);
```

```
String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);  
System.out.println("create new znode: " + newPath);
```

```
}
```

```
}
```

ZOOKEEPER:

This method is creating a znode as indicated by the user input. This znode has no value associated and is **PERSISTENT**.

PERSISTENT means that this znode will continue to exist even if the process that created it stops (or crashes).

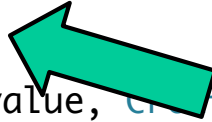
```
public class Zookeeper {
    // Main just for testing
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide a path:");
        String path = sc.nextLine().trim();

        System.out.println("Provide a value:");
        String value = sc.nextLine().trim();
        sc.close();

        ZookeeperProcessor zk = new ZookeeperProcessor("localhost:2181,kafka:2181");
        zk.write(path, null, CreateMode.PERSISTENT);

        String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);
        System.out.println("create new znode: " + newPath);
    }
}
```



ZOOKEEPER:

This method is creating a znode as indicated by the user input. This znode has no value associated and is **PERSISTENT**.

PERSISTENT means that this znode will continue to exist even if the process that created it stops (or crashes).

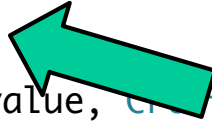
```
public class Zookeeper {
    // Main just for testing
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide a path:");
        String path = sc.nextLine().trim();

        System.out.println("Provide a value:");
        String value = sc.nextLine().trim();
        sc.close();

        ZookeeperProcessor zk = new ZookeeperProcessor("localhost:2181,kafka:2181");
        zk.write(path, null, CreateMode.PERSISTENT);

        String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);
        System.out.println("create new znode: " + newPath);
    }
}
```



Since znodes must have a unique name, this method might fail if the znode already exists (the write method returns null in that case otherwise it returns the name of the created znode).

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperWriter (Example with Main)

```
public class ZookeeperWriter {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Provide a path");  
        String path = sc.nextLine().trim();
```

```
        System.out.println("Provide a value");  
        String value = sc.nextLine().trim();  
        sc.close();
```

```
        ZookeeperProcessor zk = new ZookeeperProcessor("localhost:2181", "localhost:2181");  
        zk.write(path, null, CreateMode.PERSISTENT);
```

```
        String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);  
        System.out.println("create new znode: " + newPath);
```

```
    }
```

```
}
```

This method is creating a child znode of the previous one (hence its path starts with the name of the previous znode).

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperWriter (Example with Main)

```
public class ZookeeperWriter {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in)  
  
        System.out.println("Provide a path  
        String path = sc.nextLine().trim();  
  
        System.out.println("Provide a value  
        String value = sc.nextLine().trim()  
        sc.close();  
  
        ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");  
        zk.write(path, null, CreateMode.PERSISTENT);  
  
        String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);  
        System.out.println("create new znode: " + newPath);  
    }  
}
```

It is also associating a value to this znode, meaning that the Zookeeper server will store the contents of the value associated to this znode.

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperWriter (Example with Main)

```
package lab11.zookeeper;
```

```
import java.util.Scanner;
```

```
public class ZookeeperWriter {
```

```
    // Main just for testing
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Enter path: ");
```

```
        String path = sc.nextLine();
```

```
        System.out.println("Enter value: ");
```

```
        String value = sc.nextLine();
```

```
        sc.close();
```

```
        ZookeeperProcess zk = new ZookeeperProcess();
```

```
        zk.write(path, null, CreateMode.PERSISTENT);
```

```
        String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);
```

```
        System.out.println("create new znode: " + newPath);
```

```
    }
```

```
}
```

The creation mode is EPHEMERAL_SEQUENTIAL.

There are two implications here:

EPHEMERAL: Means that this znode will only exist while the connection of this process to zookeeper remains open. If this process closes its connection, terminates, or crashes, Zookeeper will automatically remove this znode.

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

SEQUENTIAL: Means that the name of the znode will be generated by Zookeeper by appending a sequential number to the end of the znode name provided in this call. This makes sure that a znode will always be possible to create.

The numbers used to name znodes that are meanwhile removed are not reused.



```
ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");  
zk.write(path, null, CreateMode.PERSISTENT);
```



```
String newPath = zk.write(path + "/node_", value, CreateMode.EPHEMERAL_SEQUENTIAL);  
System.out.println("create new znode: " + newPath);
```

```
}
```

```
}
```

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperProcessor (Preamble)

```
public class ZookeeperProcessor implements Watcher {
    static {
        // summarizes the logging format
        System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s");
        System.setProperty("java.util.logging.level", "ERROR");
    }
    private ZooKeeper zk;

    /**
     * @param hostPort a string in the format host:port to connect to zookeeper
     */
    ZookeeperProcessor( String hostPort) throws Exception {
        zk = new ZooKeeper(hostPort, 3000, this);
    }
}
```

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperProcessor (Preamble)

```
public class ZookeeperProcessor implements Watcher {  
    static {  
        // summarizes the logging format  
        System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s");  
        System.setProperty("java.util.logging.level", "ERROR");  
    }  
    private ZooKeeper zk;  
  
    /**  
     * @param hostPort a string in the format host:port to connect to zookeeper  
     */  
    ZookeeperProcessor( String hostPort) throws Exception {  
        zk = new ZooKeeper(hostPort, 3000, this);  
    }  
}
```

This static code is here just to modify the format of the logs generated by the Zookeeper client.

This is the constructor of the ZookeeperProcessor class.

It instantiates a Zookeeper class with `Class ZookeeperProcessor (Preamble)` that is used to interact with the Zookeeper server.

The first argument is the String containing server information, the second is the session timeout (in milliseconds) and the third is a class that implements the interface Watcher.

```
* @param hostPort a string in the format host:port to connect to zookeeper
*/
ZookeeperProcessor( String hostPort) throws Exception {
    zk = new ZooKeeper(hostPort, 3000, this);
}
```

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperProcessor (Preamble)

```
public class ZookeeperProcessor implements Watcher {  
    static {  
        // summarizes the logging format  
        System.setProperty("java.util.logging.SimpleFormatter.format", "%4$s: %5$s");  
        System.setProperty("java.util.logging.level", "ERROR");  
    }  
    private ZooKeeper zk;  
  
    /**  
     * @param hostPort a string in the format host:port to connect to zookeeper  
     */  
    ZookeeperProcessor( String hostPort) throws Exception {  
        zk = new ZooKeeper(hostPort, 3000, this);  
    }  
}
```

The Watcher interface has a single method:
`void process(WatchedEvent event)`

This will be called to handle asynchronous events issued by Zookeeper (for instance when the contents/children of a watched znode change).

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperProcessor (write method)

```
public String write( String path, String value, CreateMode mode) {  
    try {  
        return zk.create(path, (value == null ? null : value.getBytes()),  
                           ZooDefs.Ids.OPEN_ACL_UNSAFE, mode);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperProcessor (write method)

```
public String write( String path, String value, CreateMode mode) {  
    try {  
        return zk.create(path, (value == null ? null : value.getBytes()),  
                           ZooDefs.Ids.OPEN_ACL_UNSAFE, mode);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

This method creates a znode.

Note that since associating data to a znode is optional the value argument can be null.

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperProcessor (write method)

```
public String write( String path, String value, CreateMode mode) {  
    return zk.create(path, (value == null ? null : value.getBytes()),  
        ZooDefs.Ids.OPEN_ACL_UNSAFE, mode);  
} catch (KeeperException | InterruptedException e) {  
    e.printStackTrace();  
    return null;  
}  
}
```

To create a znode we use the create method of the Zookeeper class.

The arguments are (in order):

- i) the name of the znode (complete name must start with /)
- ii) the value to associate with the znode (or null if no value is to be associated)
- iii) the access control list of the znode (here we use the UNSAFE default that allows everyone to read and modify the znode)
- iv) the mode associated with the created znode (PERSISTENT, EPHEMERAL, PERSISTENT_SEQUENTIAL, or EPHEMERAL_SEQUENTIAL)

ZOOKEEPER: PROGRAMMING (CREATE A TUPLE)

Class ZookeeperProcessor (write method)

```
public String write( String path, String value, CreateMode mode) {  
    zk.create(path, (value == null ? null : value.getBytes()),  
              ZooDefs.Ids.OPEN_ACL_UNSAFE, mode);  
    catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

To create a znode we use the create method of the Zookeeper class.

The arguments are in order:

- i) the name of the znode (complete name must start with /)
- ii) the value to associate with the znode (or null if no value is to be associated)
- iii) the access control list of the znode (here we use the UNSAFE default that allows everyone to read and modify the znode)
- iv) the mode associated with the created znode (**PERSISTENT**, **EPHEMERAL**, **PERSISTENT_SEQUENTIAL**, or **EPHEMERAL_SEQUENTIAL**)

GOALS

By the end of this lab, you should be able to:

- In relation to Kafka
 - Understand the abstractions provided by Kafka
 - Understand how to publish events in Java
 - Understand how to subscribe event in Java
 - Understand the asynchronous processing of Kafka received operations in the context of replication in the project.
- **Zookeeper**
 - Understand the abstractions provided by Zookeeper
 - **Understand how to create a tuple (znode) in Java**
 - Understand how to monitor a tuple (znode) in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- Understand the implementation path for the different implementation alternatives for Replication in the second project.

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperGetChildren

```
public class ZookeeperGetChildren {
    // Main just for testing purposes
    public static void main( String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.println("Provide a path (should start with /) :");
        String path = sc.nextLine().trim();

        final ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");
        zk.getChildren( path, new Watcher() {
            @Override
            public void process(WatchedEvent event) {
                List<String> lst = zk.getChildren( path, this);
                for(String str: lst)
                    System.out.println(str);
                System.out.println();
            }
        });
        System.out.println("Enter for stop observing changes :");
        sc.nextLine().trim();

        sc.close();
    }
}
```

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperGetChildren

```
public class ZookeeperGetChildren {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with /) :");  
        String path = sc.nextLine().trim();  
  
        final ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");  
        zk.getChildren( path, new Watcher() {  
            @Override  
            public void process(WatchedEvent event) {  
                List<String> lst = zk.getChildren( path, this);  
                for(String str: lst)  
                    System.out.println(str);  
                System.out.println();  
            }  
        });  
        System.out.println("Enter for stop");  
        sc.nextLine().trim();  
  
        sc.close();  
    }  
}
```

This is a simple example that reads the children of a znode (whose name is indicated by the user) and keeps monitoring that znode for changes.

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperGetChildren

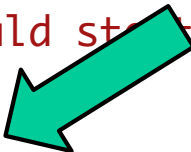
```
public class ZookeeperGetChildren {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with /):");  
        String path = sc.nextLine().trim();  
  
        final ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181");  
        zk.getChildren( path, new Watcher() {  
            @Override  
            public void process(WatchedEvent event) {  
                List<String> lst = zk.getChildren( path, this);  
                for(String str: lst)  
                    System.out.println(str);  
                System.out.println();  
            }  
        });  
        System.out.println("Enter for stop observing changes :");  
        sc.nextLine().trim();  
  
        sc.close();  
    }  
}
```

This is simple code for asking the user to provide the name of the znode to read and to monitor.

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperGetChildren

```
public class ZookeeperGetChildren {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with /) :");  
        String path = sc.nextLine().trim();  
  
        final ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");  
        zk.getChildren( path, new Watcher() {  
            @Override  
            public void process(WatchedEvent event) {  
                List<String> lst = zk.getChildren( path, this);  
                for(String str: lst)  
                    System.out.println(str);  
                System.out.println();  
            }  
        });  
        System.out.print(sc.nextLine().trim());  
  
        sc.close();  
    }  
}
```



The ZookeeperProcessor class is an auxiliary class that we will use to interact with Zookeeper (we will see the rest of the code in a bit).

It is the same we used in the previous example.

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperGetChildren

```
public class ZookeeperGetChildren {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with /) :");  
        String path = sc.nextLine().trim();  
  
        final ZookeeperProcessor zk = new ZookeeperProcessor( "localhost:2181,kafka:2181");  
        zk.getChildren( path, new Watcher() {  
            @Override  
            public void process(WatchedEvent event) {  
                List<String> lst = zk.getChildren( path, this);  
                for(String str: lst)  
                    System.out.println(str);  
                System.out.println();  
            }  
        });  
    }  
}
```



The method `getChildren` reads a List containing the name of all child znodes of the znode whose name is provided in `path` (the return value is being ignored).

The second argument is a class that implements the `Watcher` interface (defined here inline).

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperGetChildren

```
public class ZookeeperGetChildren {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Except  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with  
        String path = sc.nextLine().trim();  
  
        final ZookeeperProcessor zk = new ZookeeperProcessor(  
        zk.getChildren( path, new Watcher() {  
            @Override  
            public void process(WatchedEvent event) {  
                List<String> lst = zk.getChildren( path,  
                for(String str: lst)  
                    System.out.println(str);  
                System.out.println();  
            }  
        });  
        System.out.println("Enter for stop observing changes  
        sc.nextLine().trim();  
  
        sc.close();  
    }  
}
```

Whenever a change occurs to the contents of the watched znode (including changes in its children) an event will be issued by zookeeper, and the process method of the watcher will be executed.

In this example the watcher only reads and prints the list of child znodes.

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperGetChildren

```
public class ZookeeperGetChildren {  
    // Main just for testing purposes  
    public static void main( String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Provide a path (should start with /) :");  
        String path = sc.nextLine();  
  
        final ZookeeperProcessor zk = new ZookeeperProcessor();  
        zk.getChildren( path, new Watcher() {  
            @Override  
            public void process() {  
                List<String> children = zk.getChildren(path, false);  
                for(String s : children) {  
                    System.out.println(s);  
                }  
            }  
        });  
        System.out.println("Enter for stop observing changes :");  
        sc.nextLine().trim();  
  
        sc.close();  
    }  
}
```

The application blocks waiting for user input, so that you can execute the example and see the changes on the child znodes as writers start and stop (creating new znodes).

1");



}

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperProcessor (read methods)

```
public List<String> getChildren( String path, Watcher watch) {  
    try {  
        return zk.getChildren(path, watch);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

```
public List<String> getChildren( String path) {  
    try {  
        return zk.getChildren(path, false);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

```
@Override  
public void process(WatchedEvent event) {  
    System.out.println(event);  
}
```

```
}
```

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperProcessor (read methods)

```
public List<String> getChildren( String path, Watcher watch) {  
    try {  
        return zk.getChildren(path, watch);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

```
public List<String> getChildren( String path) {  
    try {  
        return zk.getChildren(path, false);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

```
@Override  
public void process(WatchedEvent event) {  
    System.out.println(event);  
}
```

These are the methods to read the childs of a znode and a default Watcher (that only prints the event received from Zookeeper).

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperProcessor (read methods)

```
public List<String> getChildren( String path, Watcher watch) {  
    try {  
        return zk.getChildren(path, watch);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

```
public List<String> getChildren( String path) {  
    try {  
        return zk.getChildren(path, false);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

@Override

The only difference between the two variants of the getChildren method is the existence of a Watcher, that indicates to Zookeeper if we want to be notified of changes in the read znode or not.

ZOOKEEPER: PROGRAMMING (READING CHILDS OF TUPLE)

Class ZookeeperProcessor (read methods)

```
public List<String> getChildren( String path, Watcher watch) {
```

```
    try {  
        return zk.getChildren(path, watch);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

```
public List<String> getChildren( String path) {
```

```
    try {  
        return zk.getChildren(path, false);  
    } catch (KeeperException | InterruptedException e) {  
        e.printStackTrace();  
        return null;  
    }  
}
```

```
@Override
```

```
public void process(WatchedEvent event) {
```

```
    System.out.println(event);  
}
```

```
}
```

To read the children of a znode we use the `getChildren` method of the Zookeeper class.

The arguments are: i) the name of the znode to read (must start with /)
ii) the watcher to receive notifications or false if no monitoring of the znode is necessary

ZOOKEEPER: PROGRAMMING (EXECUTING EXAMPLES)

You can (and should) test the examples provided in your machine.

To do so, follow these steps:

1. Start the kafka container as show before in localhost mode.
2. Launch one Writer and provide a znode (e.g., /SD) just to create that znode (the value is irrelevant).
3. Launch the GetChildren and provide the same znode. The process will be watching for changes.
4. Launch another Writer and provide the same znode as you did initially.
5. You can execute the Writer more times to see more changes and increasing sequence numbers.

GOALS

By the end of this lab you should be able to:

- Kafka
 - Understand the abstractions provided by Kafka
 - Understand how to publish events in Java
 - Understand how to subscribe event in Java
 - Understand the asynchronous processing of Kafka received operations in the project.
- **Zookeeper**
 - Understand the abstractions provided by Zookeeper
 - Understand how to create a tuple in Java
 - Understand how to monitor a tuple in Java
 - **Understand how Zookeeper can be used to (dynamically) elect a primary**
- Overall Revision of the Project Goals

ZOOKEEPER: LEADER ELECTION

The idea is that you will use Zookeeper (if you select that option) to elect the primary of a set of replicas for a given domain (and to pick a new one when the previous primary fails).

How can you do this:

1. Each domain should have a persistent znode with the name being the domain itself.
2. Each replica, when it starts, creates a child znode on its domain znode for itself. The child znode should be ephemeral and have names with a sequential identifier.
3. Every replica reads and watches the znode for their domain. The primary replica is the one whose child znode is the first within the domain znode.
4. You can store control information in znodes if you need.

GOALS

By the end of this lab you should be able to:

- Kafka
 - Understand the abstractions provided by Kafka
 - Understand how to publish events in Java
 - Understand how to subscribe event in Java
 - Understand the asynchronous processing of Kafka received operations in the project.
- Zookeeper
 - Understand the abstractions provided by Zookeeper
 - Understand how to create a tuple in Java
 - Understand how to monitor a tuple in Java
 - Understand how Zookeeper can be used to (dynamically) elect a primary
- **Overall Revision of the Project Goals**

GOALS OF REPLICATION

As the name implies, replication entails having replicas (multiple copies) of a service. The idea is that these replicas cooperate and coordinate among themselves to provide a service to users.

- Usually the user does not notice the existence of replicas (Transparency).
- If one replica fails, the service can continue without the user noticing the failure of one replica (Fault-Tolerance).
- Sometimes it is possible to take advantage of the additional computational resources (across multiple replicas) to handle more client requests or handle these requests faster (Performance).

PROJECT REVISION AND EXERCISE

1. Use what you have learned in this class to complete your project by supporting replication in your original REST servers.

Remember that you have multiple implementation alternatives as specified in the project web page:

- Replication using Kafka
- Replication based on Primary/Secondary (with a fixed Primary)
- Replication based on Primary/Secondary using Zookeeper to select the Primary

PROJECT REVISION AND EXERCISE

1. Use what you have learned in this class to complete your project by supporting replication in your original REST Content server.

Remember that you have multiple implementation alternatives as specified in the project web page:

- Replication using Kafka
- Replication based on Primary/Secondary (with a fixed Primary)
- Replication based on Primary/Secondary using Zookeeper to select the Primary

PICK ONLY ONE OF THE
REPLICATION ALTERNATIVES